

Politecnico di Torino

Politecnico di Torino

Bachelor degree in Management Engineering

A.a. 2025/2026

Graduation Session September 2026

A comparison of best-fit and first-fit Extreme Point heuristics for the constrained Three-Dimensional Bin Packing Problem

with an analysis of the sensitivity to item ordering

Supervisor: Fabio Salassa

Candidate: Riccardo Quariglio

Table of Contents

1 Introduction to combinatorial optimization and packing problems 1
- 1.1 Historical roots . . . 1
- 1.2 Families of problems . . . 2
- 1.3 Complexity and solving strategies . . . 2
- 1.4 Cutting and packing problems . . . 3
- 1.5 From one to three dimensions . . . 4
- 1.6 Three-dimensional bin packing in logistics . . . 4
- 1.7 Objectives and structure of the thesis . . . 5

2 Literature review 6
- 2.1 Exact and constructive approaches . . . 6
- 2.2 Metaheuristic approaches . . . 7
- 2.3 Placement rules and the extreme point concept . . . 7
- 2.4 Additional constraints and positioning of this work . . . 8

3 Problem description 9
- 3.1 Mathematical notation: parameters, indexes and variables . . . 9
- 3.2 Rotations . . . 11
- 3.3 Constraints and objective function . . . 12

4 Solution approach 16
- 4.1 Instances and input data . . . 16
  - 4.1.1 Generation of the instances . . . 17
  - 4.1.2 Reproducibility and the random seed . . . 18
  - 4.1.3 The two vehicles . . . 18
- 4.2 Data structures . . . 19
- 4.3 The extreme point rule . . . 19
  - 4.3.1 Generation of new extreme points . . . 20
  - 4.3.2 Residual space update . . . 22
- 4.4 Item sorting . . . 23
- 4.5 Feasibility check . . . 23

4.5.1 The two variants of the check . . . 24
4.6 The merit function: residual space criterion . . . 25
4.7 The two heuristics: best-fit and first-fit . . . 25
4.8 Randomized item selection . . . 27
4.9 Solution visualization and verification . . . 27
4.9.1 Solution checker . . . 28
4.10 Automation of the experiments . . . 28

5 Results and conclusions 30
5.1 Experimental setup . . . 30
5.2 Objective value analysis . . . 32
5.3 Computational time analysis . . . 33
5.4 The effect of the vehicle size . . . 34
5.5 Distance from the lower bounds . . . 36
5.6 Sensitivity to item ordering . . . 37
5.7 Packing visualization . . . 38
5.8 Summary of the results . . . 39
5.9 Managerial implications . . . 39
5.10 Limitations and future work . . . 40

A Scripts 42
A.1 Instance generation . . . 42
A.2 Data loading . . . 45
A.3 Core packing routines . . . 46
A.4 Extreme point management . . . 50
A.5 The packing loop . . . 55
A.6 Experiment automation . . . 58

List of Tables 62

List of Figures 63

Bibliography 65

Chapter 1

Introduction to combinatorial optimization and packing problems

Combinatorial optimization is the branch of operations research that looks for the best element of a finite, but usually enormous, set of alternatives. A problem of this kind is described by four ingredients:

- a set of *decision variables*, which represent the choices to be made;
- a *domain* for each variable, listing the values it may take;
- a set of *constraints*, which rule out the combinations of values that are not acceptable;
- an *objective function*, which assigns a value to every acceptable combination and has to be minimized or maximized.

What distinguishes combinatorial optimization from continuous optimization is that the domains are discrete: a lorry is either sent out or it is not, a parcel goes on one vehicle or on another. The number of combinations is finite, so in principle one could enumerate them all; in practice that number grows so quickly with the size of the problem that enumeration is hopeless even for instances of modest size, which is why the field exists at all.

1.1 Historical roots

The discipline was born from concrete industrial questions rather than from abstract mathematics. Its first formulation is usually attributed to Kantorovich, who in 1939 studied how to organise production so as to use the available machinery in the most efficient way, and whose work reached the Western literature only two

decades later [1]. The problems he described – distributing work among machines, cutting raw material with as little waste as possible – are recognisably the ancestors of what is studied today.

The direct ancestor of packing problems is the cutting stock problem, addressed by Gilmore and Gomory at the beginning of the sixties [2]. Given rolls of paper of a standard width and a list of orders for narrower rolls, the question is how to cut the stock so as to satisfy the orders while wasting as little material as possible. Their contribution was twofold: they wrote the problem as a linear program with an astronomical number of variables, and they showed how to solve it without ever writing all of them down, generating only the ones that turn out to be useful. The problem they solved is one-dimensional, but the question they asked – how to fit a set of pieces into as few containers as possible – is exactly the one this thesis addresses in three dimensions.

1.2 Families of problems

Combinatorial optimization problems are usually grouped into a few large families, according to the structure of the decisions they involve. *Assignment* problems match the elements of one set with those of another, as when tasks are assigned to workers. *Transportation* and *routing* problems move goods across a network at minimum cost, the travelling salesman problem being the best known member of the family. *Scheduling* problems place activities in time, deciding what is processed when and on which machine. *Covering* problems select a minimum number of elements so that every requirement is met by at least one of them.

Packing problems, the family this work belongs to, distribute a set of objects among containers subject to capacity restrictions. They are the mirror image of covering problems: instead of making sure that everything is covered at least once, one makes sure that nothing is used more than its capacity allows. The boundaries are not sharp, and real applications sit across several families: loading vehicles is a packing problem, but deciding the order in which those vehicles visit their customers is a routing problem.

1.3 Complexity and solving strategies

The difficulty of these problems is not a matter of insufficient computing power. Most of them belong to the class of NP-hard problems, for which no algorithm is known that solves every instance in a time growing polynomially with its size, and for which the existence of such an algorithm is considered unlikely [3]. The consequence is practical: the time needed by an exact method grows so fast that

adding a few items to an instance can turn a computation of seconds into one of days.

Two strategies are therefore available. *Exact methods*, such as branch and bound or integer programming, guarantee that the solution they return is optimal; the price is that they can only be applied to instances of limited size, and that one cannot predict how long they will take. *Heuristic methods* give up the guarantee and look for a good solution in a predictable, usually short, time. Among them one distinguishes constructive heuristics, which build one solution through a sequence of greedy decisions and never go back, from metaheuristics, which explore many solutions by repeatedly modifying them and can escape a local optimum at the cost of a much longer computation.

The interest in heuristics is not merely pragmatic. Johnson showed, in one of the founding papers of the field, that simple rules can be analysed rigorously and that their worst-case behaviour can be bounded, so that one may know in advance how far from the optimum a heuristic solution can possibly be [4]. For bin packing in one dimension these bounds are classical results: the first-fit decreasing rule, for instance, never uses much more than a fixed fraction above the minimum number of bins [5]. In higher dimensions no comparable guarantee is available, and the quality of a heuristic has to be assessed experimentally, which is what Chapter 5 does.

1.4 Cutting and packing problems

Cutting and packing problems form a family so wide that classifying it has been a research topic in itself. The typology of Wäscher, Haussner and Schumann organises it along a few criteria: the number of dimensions, the kind of assignment between objects and containers, and how varied the containers and the items are [6].

The distinction that matters most here is the one between the two kinds of assignment. In an *input minimization* problem all the small items have to be accommodated, and the objective is to use as little container material as possible: this is the bin packing problem, and it is the problem of this thesis. In an *output maximization* problem the container capacity is fixed and insufficient, so only a subset of the items can be accommodated and the objective is to select the most valuable one: this is the knapsack problem. The two are often confused because they share the same geometry. They answer opposite questions, though, and an algorithm designed for one is not directly usable for the other.

1.5 From one to three dimensions

In one dimension an item is characterised by a single number, its size, and a bin by its capacity. A solution is simply a partition of the numbers into groups whose sums do not exceed the capacity: there is no geometry, and once the group is decided nothing else remains to be said.

In two dimensions each item is a rectangle, and knowing which bin it belongs to is no longer enough: one must also say *where* in the bin it goes, and whether it is rotated. Two solutions that assign the same items to the same bins may be one feasible and the other not, depending on how the rectangles are arranged.

The step to three dimensions adds another coordinate and, with it, a qualitatively new difficulty: an item may be placed on top of another, so the positions available at any moment depend on the shape of the pile built so far, and that shape changes at every placement. In one dimension the placement rule does not exist; in two it matters; in three it becomes the decisive component of any heuristic, because the free space inside a partially loaded bin is a complicated region and how one chooses to describe it determines which positions the algorithm will ever consider [7]. This is the reason why so much of the literature on three-dimensional packing is, in substance, literature on how to represent free space.

1.6 Three-dimensional bin packing in logistics

The industrial relevance of the problem is immediate. Every vehicle that leaves a depot costs a driver, a route and a fixed amount of fuel, largely independent of how full it is, so the difference between a good and a mediocre loading plan is measured in vehicles saved, and therefore in journeys not made. In a distribution operation that dispatches the same goods every day, one vehicle less per round is a permanent saving.

The pure three-dimensional bin packing problem, however, is a geometric abstraction, and a loading plan that is optimal on paper may be unusable in practice. The review by Bortfeldt and Wäscher counts more than a dozen requirements that practitioners impose, from weight limits and weight distribution to stability and orientation restrictions [8]. Most published algorithms address only a few of them, and those concerning stability and load bearing are among the least frequently treated, although they decide whether a plan can be executed at all.

Three of these requirements are considered in this work. Goods have a weight, and a vehicle has a payload it cannot exceed. Goods have a commercial value, and an operator may be unwilling to concentrate too much value on a single vehicle. Goods have a fragility, and a parcel stacked on others needs a sufficient surface underneath it. To these must be added the restrictions on orientation: a boxed

screen or a bottled liquid cannot travel upside down, whatever the geometry would allow.

1.7 Objectives and structure of the thesis

The objective of this thesis is to implement and compare experimentally two constructive heuristics for a three-dimensional bin packing problem subject to the constraints just described. Both are based on the extreme point rule of Crainic, Perboli and Tadei [7] and differ only in how they choose among the admissible positions of an item: the first evaluates them all and keeps the one that wastes the least space, the second stops at the first it finds. The comparison is carried out along two dimensions, the quality of the solution, measured by the number of vehicles used, and the computational cost, measured by the running time, on 50 purpose-built instances and on two vehicles of markedly different size. The question the experiments are meant to answer is whether the more careful of the two rules is worth its cost, and whether the answer changes with the size of the vehicle.

The rest of the work is organised as follows. Chapter 2 reviews the literature on the three-dimensional bin packing problem, on the heuristics proposed for it and on the additional constraints that arise in practice. Chapter 3 describes the problem addressed here, fixes the notation and states the mathematical model. Chapter 4 describes the solution approach: the instances and the two vehicles used in the experiments, the extreme point rule and the data structures that support it, the two variants of the heuristic and the tools built around them for visualisation, verification and automation. Chapter 5 reports and discusses the experimental results, and draws the conclusions of the work. Appendix A collects the relevant parts of the source code.

Chapter 2

Literature review

This chapter reviews only the strands of a vast literature that directly motivate the approach adopted here; comprehensive surveys are available for the reader who wants more [6, 9, 8].

2.1 Exact and constructive approaches

The reference formulation of the problem is due to Martello, Pisinger and Vigo, who proposed an exact branch-and-bound algorithm together with lower bounds and a family of benchmark instances that later work has kept using [10]. Exact methods have improved since then, but the instances solvable to optimality remain one or two orders of magnitude smaller than the loading problems of a distribution operation, where a single round can involve several hundred parcels. This is why heuristics dominate the field, and why they are used here.

The classical constructive scheme comes from one-dimensional bin packing: both its variants sort the items by non-increasing size and place them one at a time, but the first fit decreasing rule puts each item into the first bin where it fits, while the best fit decreasing rule puts it where it fits best according to the residual capacity, and both stay within a constant factor of the optimum in one dimension [5, 4]. In three dimensions the two rules keep their names but change their nature, because the residual capacity of a bin is not a number but a shape and the same item may fit in several positions within the same bin: the decision concerns the position and not only the bin, and the difference between the rules becomes a difference between evaluating all the admissible positions and stopping at the first one. It is this pair that the present work implements and compares, in the form in which Crainic et al. define it for the extreme point rule [7].

2.2 Metaheuristic approaches

A constructive heuristic is often the starting point of a longer search: a guided local search that removes one bin at a time and moves items to resolve the resulting overlaps [11], a two-level tabu search separating the assignment of items to bins from the packing of each single bin [12], a biased random-key genetic algorithm whose chromosome encodes the order in which items are considered [13], a GRASP built on a constructive block heuristic [14], and applications of the same ideas to real industrial scenarios [15]. All of them obtain better solutions than a purely constructive rule, at a computational cost higher by orders of magnitude. The present work deliberately stops at the constructive stage: no improvement phase is applied and no solution is revised once built, so what follows compares two placement rules and makes no claim about the best solutions attainable on these instances.

2.3 Placement rules and the extreme point concept

The central question of any constructive heuristic in three dimensions is which positions an item may be given. Enumerating every point of the bin is out of the question, so the free space is described by a finite list of candidate positions, and the way that list is built determines what the algorithm can and cannot find. The oldest answer is the corner point rule, which offers the corners of the envelope of the items already loaded: cheap, but blind to the space enclosed between items, since a gap opening above a low box generates no corner. The maximal space representation is more expressive, describing the free volume as a list of the largest empty boxes inside the bin [14], but those boxes overlap one another and the list has to be recomputed and pruned at every placement.

The extreme point rule of Crainic, Perboli and Tadei sits between the two [7, 16]. When an item is loaded, the three corners along its faces become candidate positions, and each is also projected along the orthogonal axes, stopping on the first item it meets: the projections reach the space enclosed between items, and are computed from the item just placed alone, so the update stays local and inexpensive. Each point also carries its *residual space*, the free length available along the three axes before the first obstacle, which the authors use to rank the candidate positions. The rule says where an item may go, not which item to place or into which bin, which is why it combines naturally with additional constraints, as is done here; on the standard benchmarks its heuristics dominate the previously published constructive rules while remaining far faster than the metaheuristics above.

2.4 Additional constraints and positioning of this work

The gap between the geometric problem and an executable loading plan is filled by a set of additional constraints, reviewed systematically by Bortfeldt and Wäscher [8], whose survey observes that the literature treats them unevenly: weight limits and orientation restrictions are common, whereas stability and load bearing are addressed far less often although practitioners rank them among the most important. Stability has been formalised either mechanically, requiring the forces on each item to be balanced, or geometrically, requiring a sufficient fraction of the base of each item to rest on the items below – the form adopted here, which goes back at least to Bischoff, Janetz and Ratcliff in the context of pallet loading [17] and which Junqueira, Morabito and Yamashita formulate together with load bearing, the limit on how much weight may be stacked on top of an item [18].

Against this background, the present work makes three specific choices. The required fraction of supported base, a single parameter of the model in the works cited above, is here an attribute of the individual item, so that crockery and books travelling in the same vehicle are subject to different requirements: the constraint is tied to the fragility of the goods rather than to the loading policy, and binds unevenly, on items scattered through the load. The admissible orientations are likewise declared item by item. Finally, every instance is solved twice with vehicles whose loading volumes differ by a factor of four, so that the comparison between the two rules can be observed as the number of vehicles required changes. Publicly available benchmarks do not combine these features – the instance generator of Osaba et al. is a step in that direction [19] – which is why a dedicated set of instances was generated, as described in Section 4.1.

Chapter 3

Problem description

The problem addressed in this study takes into consideration a three-dimensional bin packing problem with additional loading constraints, arising from a local distribution context. A shipping operator has to dispatch a set of heterogeneous rectangular items, each one described by its dimensions, its weight, its commercial value and its own fragility. The items must be loaded onto vehicles of a single type, all identical to one another and available in unlimited number, so that the whole consignment leaves the depot in one round. Since every vehicle in the fleet is the same and no cost is associated with an individual trip, the quality of a loading plan is measured by how many vehicles it requires: the fewer the vehicles, the better the plan.

A loading plan is admissible only if it can actually be realised in practice. Every item must lie completely inside the loading area of the vehicle it is assigned to, with its edges parallel to the edges of the vehicle, and no two items may share the same portion of space. The total weight loaded on a vehicle cannot exceed its payload, and the total value carried cannot exceed the amount covered by the insurance policy of a single trip. Items may be turned before being loaded, but only in the orientations that the goods themselves tolerate. Finally, an item that does not rest on the floor of the vehicle must find enough support underneath it, in a proportion that depends on how fragile the item is.

3.1 Mathematical notation: parameters, indexes and variables

To move from the description of the problem to a mathematical model, it is necessary to define the notation used to represent the goods, the vehicle and the placement decisions. The origin of the reference frame is the corner of the loading area with the smallest coordinates along the three axes; the x axis runs along its

depth, the y axis along its width and the z axis along its height. An item is located by the corner of its box with the smallest coordinates, so that an item placed at (x_i, y_i, z_i) occupies the region $[x_i, x_i + \hat{d}_i] \times [y_i, y_i + \hat{w}_i] \times [z_i, z_i + \hat{h}_i]$, where the quantities marked with a hat denote the dimensions of the item after rotation. The complete notation is reported in Table 3.1.

Units. Throughout this work lengths are expressed in centimetres, weights in kilograms, values in euros and gravity strengths in percentage points. These are the units of the instance files, and no conversion is applied anywhere in the model or in the implementation, so every figure reported in Chapter 5 is expressed in them.

Table 3.1: Mathematical notation: parameters, indexes and variables

Symbol	**Description**
Parameters	
n	Number of items
w_i, d_i, h_i	Width, depth and height of item i in its reference orientation
w_i^r, d_i^r, h_i^r	Width, depth and height of item i under rotation r
ω_i	Weight of item i
ν_i	Value of item i
g_i	Gravity strength of item i (minimum supported base fraction)
R_i	Set of rotations allowed for item i, $R_i \subseteq \{0, \ldots, 5\}$
W, D, H	Width, depth and height of the vehicle loading area
Ω	Maximum payload of the vehicle
N	Maximum total value allowed on a single vehicle
K	Upper bound on the number of vehicles ($K = n$)
M	Big number
Indexes	
i, j	Items, $1 \leq i, j \leq n$
k	Vehicles (bins), $1 \leq k \leq K$
r	Rotation, $r \in \{0, \ldots, 5\}$
p	Separation direction, $1 \leq p \leq 6$
Variables	
u_k	1 if vehicle k is used, 0 otherwise
a_{ik}	1 if item i is loaded into vehicle k, 0 otherwise
ρ_{ir}	1 if item i is placed under rotation r, 0 otherwise
σ_{ij}^p	1 if items i and j are separated along direction p, 0 otherwise
(x_i, y_i, z_i)	Coordinates of the reference corner of item i
$\hat{w}_i, \hat{d}_i, \hat{h}_i$	Dimensions of item i in the orientation actually used
λ_{ij}	Area of the base of item i resting on the top face of item j

Three remarks are worth making on this notation. The first concerns the

orientation of the horizontal axes: depth is measured along x and width along y, whereas Crainic et al. [16], whose placement rule is adopted in Chapter 4, associate x with the width and y with the depth. The two conventions differ only in the name given to the two horizontal directions and describe the same geometry; the one used here is stated once and kept throughout text, equations and figures, so that they match the implementation reported in Appendix A. The second concerns K: since the vehicles are identical and available in unlimited number, an upper bound on their number is needed only to keep the index k finite, and the trivial bound $K = n$ is always valid. The third concerns $\hat{w}_i, \hat{d}_i, \hat{h}_i$: they are not data of the problem but consequences of the rotation chosen for the item, and are therefore defined by Eqs. (3.5) below.

3.2 Rotations

Each item is a rectangular box that must be loaded with its edges parallel to those of the loading area. Under this requirement the orientations available are the six permutations of the triple (w_i, d_i, h_i), numbered from 0 to 5 as follows:

$r = 0$: (w_i, d_i, h_i)	$r = 2$: (h_i, d_i, w_i)	$r = 4$: (w_i, h_i, d_i)
$r = 1$: (d_i, w_i, h_i)	$r = 3$: (d_i, h_i, w_i)	$r = 5$: (h_i, w_i, d_i)

Rotations 0 and 1 keep the item upright and only turn it around the vertical axis, exchanging width and depth; the remaining four lay it on one of its sides, bringing either the original height or the original depth to the vertical direction. Figure 3.1 shows the same parallelepiped in the six orientations.

Not every orientation is acceptable for every kind of goods, and this is the reason why each item carries its own subset of admissible rotations R_i. Three patterns occur in the instances: $R_i = \{0,1\}$ for goods that must travel upright, such as appliances, screen devices, crockery and bottled liquids; $R_i = \{0, \ldots, 5\}$ for robust goods in rigid boxes, such as books, board games and tools, which can be freely oriented; and the partial subset $R_i = \{0,1,4,5\}$ for boxed bicycles, which excludes the two orientations that would make the box rest on its narrow side. Modelling the admissible orientations item by item, rather than globally, keeps the instances close to what a real loading operation allows; how the subsets are assigned to the different classes of goods is described in Section 4.1.1.

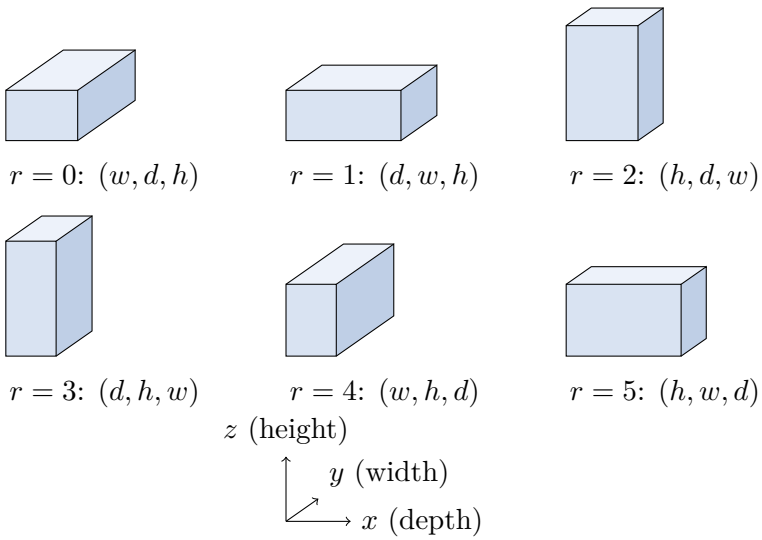


Figure 3.1: The six orthogonal orientations of an item, numbered as in the implementation. The reference triple is (w, d, h) with $w > d > h$.

3.3 Constraints and objective function

The model below states the problem formally. The non-overlapping constraints follow the classical mixed integer formulation of Chen, Lee and Shen [20], in which the relative position of each pair of items is described by binary variables and enforced through a big number; weight, value and support constraints are added to it.

Objective function

$$\min \sum_{k=1}^{K} u_k \tag{3.1}$$

Eq. (3.1) states the objective to be minimized, that is the number of vehicles used to ship the whole consignment. Because the fleet is homogeneous and no cost is attached to a single vehicle, the objective function coincides with the number of open bins: this is the metric reported throughout Chapter 5.

Constraints

$$\sum_{k=1}^{K} a_{ik} = 1 \qquad 1 \le i \le n \tag{3.2}$$

$$a_{ik} \leq u_k \qquad 1 \leq i \leq n,\ 1 \leq k \leq K \tag{3.3}$$

Eq. (3.2) forces every item to be loaded onto exactly one vehicle, so that no item is left at the depot and none is shipped twice, while Eq. (3.3) links the assignment to the objective: an item can be placed in vehicle k only if that vehicle is used, and a vehicle that receives at least one item is counted in Eq. (3.1).

$$\sum_{r \in R_i} \rho_{ir} = 1 \qquad 1 \leq i \leq n \tag{3.4}$$

$$\hat{w}_i = \sum_{r \in R_i} w_i^r \rho_{ir}, \qquad \hat{d}_i = \sum_{r \in R_i} d_i^r \rho_{ir}, \qquad \hat{h}_i = \sum_{r \in R_i} h_i^r \rho_{ir} \tag{3.5}$$

Eq. (3.4) imposes that exactly one orientation is selected for each item, and that it is selected among those the item admits, since ρ_{ir} is defined only for $r \in R_i$. Eqs. (3.5) translate that choice into the dimensions actually occupied by the item, which are the ones used by all the geometric constraints that follow.

$$x_i + \hat{d}_i \leq D, \qquad y_i + \hat{w}_i \leq W, \qquad z_i + \hat{h}_i \leq H \qquad 1 \leq i \leq n \tag{3.6}$$

Eqs. (3.6) are the containment constraints: together with the non-negativity of the coordinates, they keep every item completely inside the loading area. Since all the vehicles are identical, the same three bounds hold whatever vehicle the item is assigned to, and the index k does not appear.

$$\begin{aligned}
&x_j + \hat{d}_j \leq x_i + M(1 - \sigma_{ij}^1), && x_i + \hat{d}_i \leq x_j + M(1 - \sigma_{ij}^2), && \\
&y_j + \hat{w}_j \leq y_i + M(1 - \sigma_{ij}^3), && y_i + \hat{w}_i \leq y_j + M(1 - \sigma_{ij}^4), && 1 \leq i < j \leq n \\
&z_j + \hat{h}_j \leq z_i + M(1 - \sigma_{ij}^5), && z_i + \hat{h}_i \leq z_j + M(1 - \sigma_{ij}^6) &&
\end{aligned} \tag{3.7}$$

$$\sum_{p=1}^{6} \sigma_{ij}^p \geq a_{ik} + a_{jk} - 1 \qquad 1 \leq i < j \leq n,\ 1 \leq k \leq K \tag{3.8}$$

Eqs. (3.7) and (3.8) express the non-overlapping requirement. Each variable σ_{ij}^p activates one of the six ways in which two boxes can be kept apart, that is one item entirely on one side of the other along one of the three axes: when $\sigma_{ij}^p = 1$ the corresponding inequality separates them, when $\sigma_{ij}^p = 0$ the big number M, taken larger than any dimension of the loading area, makes that inequality redundant. Eq. (3.8) then requires at least one separation direction to be active whenever items i and j share the same vehicle, that is when $a_{ik} = a_{jk} = 1$, and leaves the pair unconstrained otherwise.

$$\sum_{i=1}^{n} \omega_i \, a_{ik} \leq \Omega \, u_k \qquad 1 \leq k \leq K \tag{3.9}$$

$$\sum_{i=1}^{n} \nu_i \, a_{ik} \leq N \, u_k \qquad 1 \leq k \leq K \tag{3.10}$$

Eq. (3.9) is the payload constraint, which prevents the goods loaded onto a vehicle from exceeding the weight it is allowed to carry, and Eq. (3.10) is its economic counterpart, which caps the total value travelling on a single vehicle. The latter has no geometric meaning but is a common operational requirement, since the insurance coverage of a shipment is usually stipulated per vehicle: splitting a consignment of high-value goods over more vehicles limits the exposure to a single accident or theft. Both are written with u_k on the right-hand side, so that an unused vehicle is forced to remain empty.

The gravity strength constraint

The last requirement is the one that distinguishes this model from the pure three-dimensional bin packing problem: an item stacked on top of others must be adequately supported from underneath. Support is measured as the fraction of the base area of the item that rests on the top faces of the items immediately below it. Let λ_{ij} be the contact area between item i and item j, counted only when the top face of j is coplanar with the base of i:

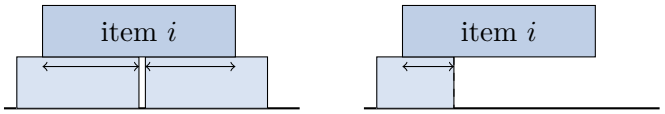
$$\lambda_{ij} = \begin{cases} \Big[\min(x_i + \hat{d}_i,\, x_j + \hat{d}_j) - \max(x_i,\, x_j)\Big]^{+} & \\ \qquad \cdot \Big[\min(y_i + \hat{w}_i,\, y_j + \hat{w}_j) - \max(y_i,\, y_j)\Big]^{+}, & \text{if } z_j + \hat{h}_j = z_i, \\ 0, & \text{otherwise,} \end{cases} \tag{3.11}$$

where $[\,\cdot\,]^{+} = \max(0, \cdot)$ denotes the positive part. Then, for every item i that does not rest on the floor and for the vehicle k it is assigned to:

$$\sum_{\substack{j=1 \\ j \neq i}}^{n} \lambda_{ij} \, a_{jk} \;\geq\; \frac{g_i}{100} \, \hat{w}_i \hat{d}_i \qquad 1 \leq i \leq n,\; 1 \leq k \leq K,\; a_{ik} = 1,\; z_i > 0 \tag{3.12}$$

Eq. (3.11) computes the contact area as the product of the two overlaps of the footprints of i and j along the horizontal axes, each of which is zero when the two projections are disjoint; the case distinction discards every item that is not exactly underneath, whether it is lower down in the stack or at a different height altogether.

Eq. (3.12) then states that the sum of these contact areas, taken over the items loaded onto the same vehicle, must cover at least the fraction g_i of the base area of item i. An item lying on the floor, that is with $z_i = 0$, is regarded as fully supported and is exempt from the constraint; in the implementation the equality of Eq. (3.11) is verified within a tolerance of 10^{-6}, to compensate for the rounding of floating-point coordinates. Figure 3.2 illustrates a satisfied and a violated case.



(a) supported fraction $\approx 97\% \geq g_i = 80\%$

(b) supported fraction $\approx 27\% < g_i = 80\%$

Figure 3.2: Side view of the gravity strength constraint. The support of item i is the sum of its contact areas with the items whose top face is coplanar with its base; in case (b) the placement is rejected even though the item does not overlap anything and stays inside the vehicle.

In the models that introduce a support requirement, the required fraction is usually a single parameter of the model, the same for every item on board. In this work it has deliberately been attached to the individual item instead: g_i is a datum of the goods, not of the vehicle. The reason for the choice is that a single delivery round carries things of very different fragility – a set of ceramic tableware needs to rest on an almost continuous surface, a box of books tolerates a much looser support – and a model with one global parameter forces the planner either to apply the strictest requirement to the whole load, wasting space, or to apply an average one, which no fragile item is actually protected by. Making the requirement travel with the item lets a single shipment mix both, and makes the constraint bind exactly where the goods require it, at the price of a constraint that is harder to satisfy because the binding items are scattered through the load. The values used in the instances, and the reasoning behind them, are described in Section 4.1.1.

Chapter 4

Solution approach

The problem of Chapter 3 is NP-hard, and the instances considered here are too large to be solved to optimality. The model of Section 3.3 uses six separation variables for every pair of items: on the largest instance that would mean several million binary variables. The approach adopted here is therefore a constructive heuristic. It builds one loading plan by placing an item at a time, and never reconsiders a placement once it has been made.

The heuristic is based on the *Extreme Points* of Crainic et al. [7]. Instead of considering every position inside the vehicle, it keeps a short list of candidate positions and updates it at each placement. Two variants have been implemented. The BEST-FIT variant evaluates all the admissible positions of an item and keeps the one that wastes the least space; the FIRST-FIT variant stops at the first one it finds. They correspond to the EP-BFD and EP-FFD rules of the original paper, adapted to the additional constraints of this model. Everything else is shared between them, so the comparison of Chapter 5 measures the placement rule alone.

4.1 Instances and input data

Because no public benchmark includes at the same time the value constraint and a per-item support requirement, a dedicated set of instances was generated for this study. It consists of 50 instances, named `Dataset1` to `Dataset50`, for a total of 7 130 items, ranging from 18 to 1 000 items each. Each instance is a folder containing two comma-separated files: `items.csv`, which lists the goods to be shipped one per line, and `vehicles.csv`, which describes the available vehicles and is identical in every instance. Listing 4.1 shows the first lines of an input file.

Both files are read by a dedicated class, `Instance`, which loads them into two `pandas` data frames indexed by the identifier of the item and by the name of the

```
id,width,depth,height,weight,value,allowedRotations,
   gravityStrength
I0000,12.8,9.5,4.5,0.362,709.37,012345,82.8
I0001,20.9,14.0,9.0,0.796,129.91,01,81.2
I0002,15.9,6.2,5.7,0.893,1212.46,012345,60.4
```

Listing 4.1: First lines of `items.csv` for `Dataset1`.

vehicle respectively.[1] The solver therefore never parses a file itself. The name of the vehicle to be used is not read from the file but passed explicitly to the solver, so that the same instance can be solved twice, once per vehicle, without being reloaded.

4.1.1 Generation of the instances

The items are not drawn from a uniform distribution over an arbitrary range. They are sampled from 19 product categories, each one modelled on the packaging of a real class of goods, with its own ranges for dimensions, weight, value, gravity strength and admissible rotations; the complete table is in Appendix A.1. A category fixes the size and the shape of the box together with the properties that constrain its handling, so that the correlation between them is preserved: large appliances are bulky, heavy and must stay upright, books are small, light, cheap and freely orientable. Sampling dimensions and constraints independently would have produced instances that are formally valid but physically meaningless.

Within a category, every attribute is drawn uniformly at random inside the corresponding range. The gravity strength follows the fragility of the product, from 10% for robust goods in rigid boxes up to 96% for the most breakable ones, and the admissible rotations follow the same logic: the upright-only subset goes to the categories that would spill or break if laid down, the free subset to robust boxes, and the partial subset to boxed bicycles.

The 50 instances differ from one another in two respects only: the merchandise profile and the number of items. Sixteen profiles were defined as weighted mixtures of the 19 categories, each one reproducing a plausible delivery round, and each appears in more than one instance with a different size. The number of items ranges from a few dozen, as in a local delivery, to a few hundred, as in a regional round; profile and size of every instance are reported, together with its results, in Table 5.1. Two instances, `Dataset25` with 950 items and `Dataset50` with 1 000, are deliberate limit cases: no real delivery round loads a thousand heterogeneous

[1] For the full source code, please refer to Appendix A.2.

parcels at once, but they are needed to measure how the two heuristics scale. They are framed as a warehouse return consolidation and follow the same realism rules as the others.

4.1.2 Reproducibility and the random seed

The instances are produced by a Python script that draws every attribute at random. The numbers it draws are not truly random: a pseudo-random generator turns an initial integer, the *seed*, into a sequence that has the statistical properties of a random one. Starting from the same seed reproduces the same sequence, and hence the same instances, on any machine and at any time. The seed used here is `20260804` and is fixed in the script, so anyone who runs it obtains the 50 instances of this study unchanged. The same device returns in Section 4.8, where a seed fixed per experiment makes the randomized runs repeatable. A companion script validates the generated files against the expected schema.

4.1.3 The two vehicles

Every instance is shipped with the same two vehicles, reported in Table 4.1. They are not used together: the fleet remains homogeneous, and each instance is solved twice, once assuming that the operator owns only vehicles of the first type and once assuming that it owns only vehicles of the second one. The two vehicles are therefore not an additional degree of freedom of the model but a second experimental condition, applied to an unchanged consignment.

Table 4.1: The two vehicles, identical in every instance. The short names in the first column are the ones used throughout this work.

Vehicle	**Name in the file**	W	D	H	Ω	N	**Volume**
Ducato medio	`furgone_ducato_medio`	187	312	193	1 200	60 000	11.26 m^3
Torino piccolo	`furgone_torino_piccolo`	125	182	121	660	25 000	2.75 m^3

Both are derived from the technical specifications of commercial vehicles in regular use for local distribution: the *Ducato medio* is a medium-wheelbase panel van, the standard means of transport for urban last-mile delivery, and the *Torino piccolo* is a small city van, used where narrow streets make the larger one impractical. These two names are used throughout the rest of this work. Their loading volumes stand in a ratio of about 4.1 to 1 and their payloads in a ratio of about 1.8 to 1, so the same consignment fills a markedly different number of them. A heuristic that behaves well when one vehicle swallows a large part of the consignment does not necessarily behave well when the same goods must be split over many small

vehicles, where each decision has less room to be corrected later. Running every instance under both conditions shows whether the gap between the two heuristics widens or narrows with the size of the vehicle, which is the central experimental question of Chapter 5.

4.2 Data structures

The algorithm is built around four classes. `Item` represents a single piece of goods: besides the data read from the file, it stores its current orientation, the dimensions that follow from it and the coordinates of its reference corner once placed. It also provides the geometric primitives used by the feasibility checks, that is the three axis-by-axis overlap tests and the contact area with another item, which implements Eq. (3.11).

`Container` represents one open vehicle. Besides its dimensions and its residual payload and value, it holds the two fields on which the placement rule rests: `extreme_points`, the list of the candidate positions available inside that vehicle, and `ep_residual_space`, which associates with each of them the free space still available along the three axes. A newly opened vehicle contains a single extreme point, the origin (0,0,0), whose residual space is the full size of the loading area.

`SingleSolutionFeasible` represents one candidate placement: a container, an item, the extreme point where the item would go, the rotation and the residual space of that point at the moment of the evaluation. It is what the feasibility check returns, and it carries the merit of the placement, a value computed and used only by the BEST-FIT variant (Section 4.5.1). `PackingMethods`, finally, collects the whole placement logic: loading, sorting, feasibility, packing and the maintenance of extreme points and residual spaces.

4.3 The extreme point rule

When an item is loaded into a vehicle it does not only occupy space: it also creates new places where other items can be put, along its faces and on top of it. The extreme point rule turns this observation into a data structure. Each open vehicle carries a list of positions, the extreme points, and these are the only positions where the next item may be placed. Every time an item is loaded, the point it occupied is removed from the list and new points are generated from the faces of the item just placed.

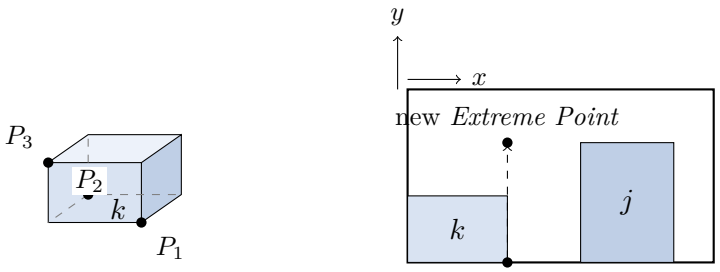
This is what distinguishes extreme points from the older corner point rule. A corner point is generated only where the envelope of the loaded items leaves a step, so the free space above a low item, or beside a narrow one, is never offered as a candidate position and remains unused. Extreme points are instead obtained by

projecting the corners of the item just loaded onto whatever lies in front of them. They therefore reach the volume enclosed between items already loaded, which is exactly the space a human loader would fill with the smaller parcels.

The rule was adopted here for a second reason as well, which is computational. Loading an item updates the list of candidate positions *locally*: one point is removed, at most nine are added, and the residual space of the surviving points is corrected against the single item just placed. Nothing is recomputed from scratch, so the description of the free space never has to be rebuilt as the vehicle fills up. The maximal-space representation describes the free volume more faithfully, but has to be recomputed and pruned at every placement. It is this cheap update that keeps an exhaustive evaluation of all the candidate positions affordable: without it the BEST-FIT variant of Section 4.7 would be out of reach on instances of a thousand items.

4.3.1 Generation of new extreme points

Let k be an item that has just been placed at (x_k, y_k, z_k) with dimensions $(\hat{w}_k, \hat{d}_k, \hat{h}_k)$. Three corners of the box are immediately new candidate positions: one beyond its far face, one beside it and one on top of it. Each of these three points is then projected along the two axes orthogonal to the one that generated it, and the projection stops on the nearest item that lies between the point and the wall of the vehicle. Six projections are therefore attempted for every item loaded, and each of them yields at most one new extreme point: the one supported by the nearest obstacle. Figure 4.1 illustrates both mechanisms.



(a) the three corners of item k

(b) projection onto the nearest item, seen from above

Figure 4.1: Generation of extreme points. In (a) the item just placed offers three new positions along its own faces, namely $P_1 = (x_k{+}\hat{d}_k,\, y_k,\, z_k)$, $P_2 = (x_k,\, y_k{+}\hat{w}_k,\, z_k)$ and $P_3 = (x_k,\, y_k,\, z_k{+}\hat{h}_k)$; in (b) one of those positions is projected along the y axis and stops on item j, the nearest obstacle, creating a point that the corner rule would not have produced.

Algorithm 1 describes the procedure. The predicate CanTakeProjection implements Algorithm 1 of the original paper [7]: for a given direction it verifies that the corner of the new item falls within the span of the candidate obstacle on the two remaining axes, and that the obstacle lies on the correct side of it. Among the items that pass the test, the one whose face is farthest along the direction of projection is the nearest to the point, and is the one retained. The three corners of the item are always added as well, which covers the case of a direction in which no other item can support a projection, in particular that of an empty vehicle. Points outside the loading area are discarded, and the list is sorted by non-decreasing z, y and x, so that positions close to the floor are examined first.

Algorithm 1 Generation of the new extreme points after a placement

```
1: procedure UpdateExtremePoints(container, k)
2:     remove from container the extreme point occupied by k
3:     UpdateResidualSpace(container, k)                          ▷ Algorithm 2
4:     for all direction δ of the six projections do
5:         best_δ ← −1; point_δ ← ∅
6:     end for
7:     for all item j already placed in container, j ≠ k do
8:         for all direction δ of the six projections do
9:             if CanTakeProjection(k, j, δ) then
10:                c ← coordinate of the face of j met by the projection
11:                if c > best_δ then
12:                    best_δ ← c; point_δ ← corner of k moved onto that face
13:                end if
14:            end if
15:        end for
16:    end for
17:    P ← {point_δ} ∪ {P_1, P_2, P_3}                    ▷ the three corners of k
18:    for all p ∈ P inside the loading area and not already in the list do
19:        rs ← (D − p_x, W − p_y, H − p_z)
20:        for all item j placed in container do
21:            rs ← ShrinkResidualSpace(p, rs, j)
22:        end for
23:        add p to the extreme points of container with residual space rs
24:    end for
25:    sort the extreme points of container by non-decreasing (z, y, x)
26: end procedure
```

4.3.2 Residual space update

Every extreme point carries its *Residual Space*: the distance, along each of the three axes, between the point and the first obstacle met in that direction, be it a wall of the vehicle or an item already loaded. It is what makes it possible to judge how well an item fits a position without re-examining the whole content of the vehicle, and it is used by the merit function of Section 4.6.

Two operations maintain it. The first is the initialisation of a new point, already shown in Algorithm 1: the residual space starts as the distance from the three walls and is then shrunk against every item already present. The second is the incremental update of Algorithm 2, which corresponds to Algorithm 2 of the paper. After a placement, the item just loaded may have become the nearest obstacle for points that were tracking something farther away, so each surviving point is shrunk against that single item. Only one item has to be examined, so the update stays inexpensive however full the vehicle is.

Algorithm 2 Incremental update of the residual space

procedure UpdateResidualSpace(*container*, k)
  for all extreme point p of *container* **do**
    $rs(p) \leftarrow$ ShrinkResidualSpace(p, $rs(p)$, k)
  end for
end procedure

function ShrinkResidualSpace(p, rs, j)
  if $z_j \le p_z < z_j + \hat{h}_j$ **then** ▷ j shares the height band of p
    if $p_x \le x_j$ **and** $y_j \le p_y < y_j + \hat{w}_j$ **then**
      $rs_x \leftarrow \min(rs_x,\ x_j - p_x)$
    end if
    if $p_y \le y_j$ **and** $x_j \le p_x < x_j + \hat{d}_j$ **then**
      $rs_y \leftarrow \min(rs_y,\ y_j - p_y)$
    end if
  end if
  if $p_z \le z_j$ **and** $x_j \le p_x < x_j + \hat{d}_j$ **and** $y_j \le p_y < y_j + \hat{w}_j$ **then**
    $rs_z \leftarrow \min(rs_z,\ z_j - p_z)$
  end if
  return rs
end function

One implementation choice deserves to be stated explicitly. The paper initialises the residual space of a new point with the distance from the walls alone, whereas the implementation described here also scans the items already loaded, as in the

last lines of Algorithm 1. The paper defines the residual space as the distance from the wall *or* from the nearest item, and a point born next to an item already there would otherwise start with an overstated free space. The incremental update would never correct it, because it only considers items loaded afterwards. The choice makes the merit function more accurate, at the price of one scan of the vehicle per new point.

4.4 Item sorting

Before any placement takes place, the items are sorted once and for all by non-increasing base area, then by height and finally by weight. Base area and height are computed over all the admissible rotations of the item, so that an item is ranked by the largest footprint it may present, not by the one of its reference orientation.

This ordering is the *decreasing* component of the two heuristics. A bulky item placed late has little chance of finding a position, while a small one fits almost anywhere, so it pays to place the large ones while the vehicle is still empty. The sorting is deterministic and identical for the two variants; Section 5.6 measures how much the result depends on it.

4.5 Feasibility check

Given an item and an open vehicle, the feasibility check enumerates the positions at which that item could be loaded. A position is a pair made of an admissible rotation and an extreme point, and it has to pass four tests, in the order in which they appear in Algorithm 3. The first two do not depend on the position: the weight and the value of the item must not exceed the residual payload and the residual value of the vehicle, Eqs. (3.9) and (3.10). If either fails, no position needs to be examined. The third is containment, Eqs. (3.6), checked against the dimensions of the item under the rotation being tried. The fourth is the absence of overlap with the items already loaded, Eqs. (3.7), tested pair by pair. The last is the gravity strength constraint, Eq. (3.12): items resting on the floor pass it by definition, while for the others the contact areas with the items already loaded are summed and compared with the required fraction of the base area.

Algorithm 3 Feasible positions of an item in an open vehicle (BEST-FIT variant)

```
1: function BestFitIsFeasible(i, container)
2:     if ω_i > residual payload or ν_i > residual value then
3:         return ∅
4:     end if
5:     S ← ∅
6:     for all rotation r ∈ R_i do
7:         set the dimensions of i to (w_i^r, d_i^r, h_i^r)
8:         for all extreme point p of container do
9:             place i at p
10:            if i exceeds the loading area then
11:                continue
12:            end if
13:            if i overlaps an item already placed then
14:                continue
15:            end if
16:            if p_z > 0 and support of i < (g_i/100) ŵ_i d̂_i then
17:                continue
18:            end if
19:            s ← candidate placement (container, i, p, r, rs(p))
20:            ComputeMerit(s)                          ▷ Section 4.6
21:            S ← S ∪ {s}
22:        end for
23:    end for
24:    restore the original rotation and position of i
25:    return S
26: end function
```

4.5.1 The two variants of the check

Algorithm 3 is the BEST-FIT version: the enumeration is exhaustive, and every position that passes the four tests is kept together with its merit. Its cost is proportional to the number of rotations times the number of extreme points, and each candidate costs a scan of the items already loaded; on a vehicle that is almost full this is the dominant cost of the whole algorithm. The reward is that the caller receives the complete set of possibilities.

The FIRST-FIT version runs through rotations and extreme points in the same order, but returns as soon as the first admissible position is found and never computes any merit. Two lines change: the call to ComputeMerit disappears, and the accumulation into S becomes an immediate return.

The two checks are written as separate functions rather than as one function with a flag. This is deliberate, not duplicated code. Had the FIRST-FIT variant called the BEST-FIT one and discarded all but the first result, it would have paid for the exhaustive enumeration anyway, and the running times compared in Chapter 5 would have measured nothing but the overhead of throwing results away.

4.6 The merit function: residual space criterion

Only the BEST-FIT variant needs to rank the positions it has enumerated, and it does so with the residual space criterion of the original paper [7]. Let (rs_x, rs_y, rs_z) be the residual space of the extreme point under evaluation and $(\hat{d}_i, \hat{w}_i, \hat{h}_i)$ the dimensions of the item in the rotation being tried. The waste of the placement is

$$f_b = (rs_x - \hat{d}_i) + (rs_y - \hat{w}_i) + (rs_z - \hat{h}_i), \tag{4.1}$$

and the merit of the placement is $-f_b$: the position with the highest merit is the one selected. Eq. (4.1) sums, over the three axes, the free length that the item leaves unused in the space available at that point. A small value means that the item nearly fills the gap it is put into, so the criterion prefers the position where the item fits most tightly. The residual space is measured against both the walls and the items already loaded, so an item is driven towards the niches left by the previous placements rather than towards the open middle of the vehicle.

4.7 The two heuristics: best-fit and first-fit

Algorithm 4 is the main loop, and it is the same for the two variants: the strategy appears in it only twice, once as the feasibility check to be called and once as the rule for choosing among the positions returned. Everything else – loading, sorting, opening a vehicle, updating extreme points, handling an item that fits nowhere – is shared.

The loop takes the items in the order established by the sorting and, for each of them, interrogates *all* the vehicles already open before deciding. This matters: an item that does not fit the vehicle being filled may still fit one opened earlier, and looking at all of them keeps the earlier vehicles from being abandoned half empty as soon as one item fails to fit. A new vehicle is opened only when no open one can take the item. The last branch, in which an item does not fit even in an empty vehicle, would leave that item behind and make the solution infeasible: it exists to guarantee termination, and is never taken on these instances, because the generator guarantees that every item fits at least one of the two vehicles in at least one orientation.

Algorithm 4 Constructive packing of an instance with a given strategy

```
 1: procedure Pack(instance, vehicle, strategy)
 2:     L ← items of instance; T ← template of vehicle
 3:     L ← SortByAreaHeightWeight(L)
 4:     V ← { a new empty copy of T }                    ▷ first vehicle opened
 5:     while L ≠ ∅ do
 6:         i ← first item of L
 7:         S ← ∅
 8:         for all container ∈ V do
 9:             S ← S ∪ strategy.IsFeasible(i, container)
10:         end for
11:         if S ≠ ∅ then
12:             s ← strategy.Select(S)          ▷ highest merit, or the first found
13:             Place(s); UpdateExtremePoints(s.container, i)
14:             remove i from L
15:         else
16:             c ← a new empty copy of T; V ← V ∪ {c}
17:             S ← strategy.IsFeasible(i, c)
18:             if S ≠ ∅ then
19:                 s ← strategy.Select(S)
20:                 Place(s); UpdateExtremePoints(c, i)
21:                 remove i from L
22:             else
23:                 close c again, discard i with a warning
24:             end if
25:         end if
26:     end while
27:     return V
28: end procedure
```

The two variants therefore differ in exactly two functions. On the same instance and the same vehicle they load the same items in the same order, until the first position where the BEST-FIT merit selects something other than the first admissible one; from there their packings diverge. Every difference measured in Chapter 5, in objective value as in computing time, comes from that single decision rule.

4.8 Randomized item selection

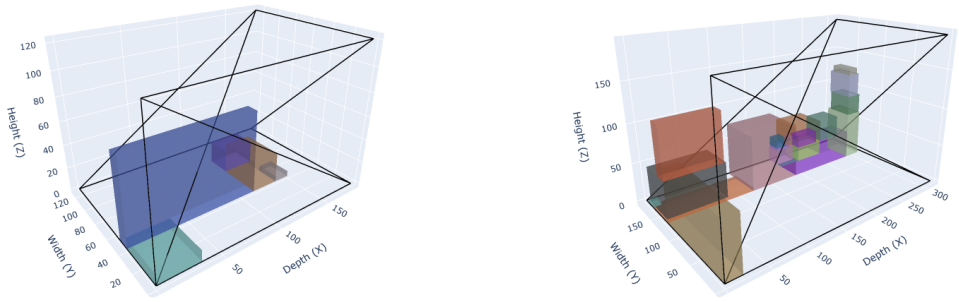
The sorting of Section 4.4 is a heuristic in itself, and the quality of the solution depends on it as much as on the placement rule. To measure how much, the main loop accepts a probability p: at every iteration, instead of taking the first item of the ordered list, with probability p it draws one uniformly at random among those still to be placed. With $p = 0$ the behaviour is exactly the deterministic one, so the same code path serves both the ordinary runs and the sensitivity experiment.

Four levels are used, $p \in \{0\%, 10\%, 20\%, 30\%\}$. The random generator is seeded per level, in the sense of Section 4.1.2, and the seed is *shared* between the two variants, so that at a given level the BEST-FIT and the FIRST-FIT runs see the same sequence of draws and process the items in the same order: whatever difference remains between them is still attributable to the placement rule alone.

4.9 Solution visualization and verification

A loading plan made of coordinates is impossible to judge by eye, so a separate module produces, for a given solution, a self-contained HTML page based on Plotly. Each open vehicle is drawn as a three-dimensional box containing the items assigned to it, in a scene the reader can rotate and zoom, and the solution is embedded in the page as a CSV fragment: the file is at the same time the picture and the data behind it. During development this was the quickest way to notice a placement that was formally admissible but visually implausible, a kind of error that no numerical check reveals as fast as a picture.

Figure 4.2 shows two examples exported from the interactive page. The black wireframe is the loading area and each solid is one item, drawn semi-transparent so that what lies behind remains visible; the axes follow the convention of Section 3.1. On the left, five items on a Torino piccolo: the tall panel standing against the side is restricted to the upright rotations, the flat one on the floor is freely orientable. On the right, twenty items on a Ducato medio show what stacking looks like when the support constraint is satisfied, with three items resting one on top of the other at the far end of the vehicle.



(a) `dataset_prova_1` on a Torino piccolo: 5 items.

(b) `dataset_prova_2` on a Ducato medio: 20 items.

Figure 4.2: Two loading plans as rendered by the visualization module. The vehicle is drawn as a wireframe and each item as a semi-transparent solid, at the coordinates and in the rotation chosen by the heuristic.

4.9.1 Solution checker

Nothing in the solver verifies the solver: the same geometric routine that places an item would validate its own mistakes. An independent checker therefore re-reads each solution from the file in which it was written and verifies, from the raw coordinates and the original instance data, that every item lies inside its vehicle, that no two items intersect, that the payload and value limits are respected, that every stacked item has the support its gravity strength requires and that no item is missing from the plan. It shares no code with the solver, so an error in the placement logic cannot hide itself from it. All 200 solutions of the campaign – fifty instances, two variants, two vehicles – were verified in this way, and none violates any constraint.

4.10 Automation of the experiments

The experimental campaign is driven by a single script.[2] For each of the 50 instances it runs 16 simulations, that is the two variants at the four levels of randomization on each of the two vehicles, and writes two files per instance: one with the objective value and the running time of the two variants in the deterministic case, one with the objective value of all 16 combinations. Running time is measured around the whole packing procedure with a performance counter, and the two variants are

[2]For the full source code, please refer to Appendix A.6.

timed in the same process, one after the other, on the same instance, so that the comparison is not affected by the environment. These files are the raw material of Chapter 5.

Chapter 5

Results and conclusions

This chapter reports the outcome of the experimental campaign. Three quantities are used throughout, and it is worth defining them before any table is read. The first is the variation of the objective value of the BEST-FIT variant with respect to the FIRST-FIT one,

$$\Delta_{obj} = \frac{n_{bf} - n_{ff}}{n_{ff}} \cdot 100, \tag{5.1}$$

where n_{bf} and n_{ff} are the numbers of vehicles used by the two variants; a negative value means that the BEST-FIT variant is the better of the two. The second is the computational overhead of the BEST-FIT variant,

$$\Delta_{time} = \frac{t_{bf} - t_{ff}}{t_{ff}} \cdot 100, \tag{5.2}$$

which is positive whenever the exhaustive evaluation costs more than stopping at the first admissible position. The third, used only in Section 5.6, is the variation of the objective value obtained when the sorting is corrupted with probability p, measured against the deterministic FIRST-FIT run on the same vehicle.

5.1 Experimental setup

All the experiments were run on a laptop equipped with an Intel Core i3-1005G1 processor at 1.20 GHz (2 physical cores, 4 threads) and 8 GB of RAM, under Windows 11, with Python 3.14.3 and the libraries `pandas` 3.0.2, `numpy` 2.4.4 and `plotly` 6.7.0. The implementation is single-threaded and no part of it is parallelised, so the running times reported below are sequential times on a single core.

The campaign covers the 50 instances described in Section 4.1, from 18 to 1 000 items, for a total of 7 130 items. Each instance was solved 16 times, namely

with the two variants, at the four levels of randomization and on each of the two vehicles, which amounts to 800 runs. The two variants are always executed in the same process, one immediately after the other, on the same instance, so that the comparison of their running times is not affected by the state of the machine. Each of the 200 solutions was then verified with the independent checker of Section 4.9.1: none of them violates the containment, overlap, weight, value or gravity strength constraints, and no item is missing from any loading plan.

Table 5.1 reports, for each instance, the profile of the consignment, its number of items and, for each of the two vehicles, the number of vehicles used and the running time of the two variants.

Table 5.1: Complete results on the 50 instances. For each vehicle: number of vehicles used (n) and running time in seconds (t) of the two variants.

			`ducato_medio`				`torino_piccolo`			
#	Profile	Items	n_{bf}	n_{ff}	t_{bf}	t_{ff}	n_{bf}	n_{ff}	t_{bf}	t_{ff}
1	Consumer electronics	25	1	1	0.04	0.01	1	1	0.05	0.01
2	Books and publishing	40	1	1	0.47	0.02	1	1	0.20	0.02
3	Mixed e-commerce	60	1	1	0.91	0.06	1	1	0.37	0.06
4	Clothing and footwear	30	1	1	0.10	0.01	1	1	0.17	0.02
5	Home and kitchen	45	1	1	0.13	0.03	2	1	0.12	0.05
6	Toys and games	55	1	1	0.92	0.04	1	1	0.35	0.07
7	Sport and leisure	35	1	1	0.16	0.01	3	2	0.06	0.02
8	DIY and tools	40	1	1	0.54	0.07	1	1	0.21	0.02
9	Food and beverages	90	1	1	1.39	0.11	2	2	0.55	0.12
10	Mixed retail distr.	120	1	1	2.89	0.37	2	2	1.18	0.43
11	Flat-pack furniture	25	1	1	0.03	0.01	2	2	0.02	0.01
12	Large appliances	18	1	1	0.02	0.01	2	2	0.01	0.01
13	Office supplies	50	1	1	0.46	0.04	2	2	0.26	0.03
14	Music and hobby	22	1	1	0.07	0.01	2	1	0.05	0.00
15	Pet shop	35	1	1	0.13	0.01	2	2	0.10	0.07
16	Consumer electronics	70	1	1	0.77	0.11	2	2	0.43	0.04
17	Books and publishing	85	1	1	1.25	0.09	1	1	0.89	0.16
18	Mixed e-commerce	100	1	1	1.76	0.10	1	1	1.47	0.09
19	Clothing and footwear	75	1	1	0.52	0.05	1	1	0.54	0.08
20	Home and kitchen	65	1	1	0.43	0.13	2	2	0.19	0.14
21	Toys and games	130	1	1	4.20	0.28	1	1	3.19	0.37
22	Sport and leisure	60	1	1	0.35	0.05	2	2	0.25	0.04
23	DIY and tools	95	1	1	1.88	0.19	2	2	0.58	0.08
24	Food and beverages	150	2	2	3.59	0.53	3	2	1.91	0.23
25	Warehouse returns	950	5	5	139.48	21.91	10	10	92.59	32.55
26	Mixed retail distr.	180	1	1	8.80	0.99	3	3	3.69	1.43
27	Flat-pack furniture	70	2	2	0.18	0.09	5	5	0.15	0.06
28	Large appliances	45	2	2	0.05	0.02	5	5	0.04	0.03
29	Office supplies	100	1	1	2.17	0.11	2	2	1.47	0.16
30	Music and hobby	40	1	1	0.27	0.02	2	1	0.26	0.03

			ducato_medio				torino_piccolo			
#	Profile	Items	n_{bf}	n_{ff}	t_{bf}	t_{ff}	n_{bf}	n_{ff}	t_{bf}	t_{ff}
31	Pet shop	80	1	1	1.10	0.16	4	3	0.48	0.27
32	Consumer electronics	110	1	1	2.87	0.17	3	3	0.77	0.08
33	Books and publishing	160	1	1	24.89	1.82	1	1	16.11	1.27
34	Mixed e-commerce	190	1	1	11.83	0.62	2	2	14.44	1.54
35	Clothing and footwear	170	1	1	7.77	0.73	1	1	5.56	0.78
36	Home and kitchen	140	1	1	2.82	0.61	3	3	1.97	1.88
37	Toys and games	210	1	1	31.69	1.05	2	2	7.36	2.09
38	Sport and leisure	150	1	1	4.70	0.42	3	3	2.37	0.99
39	DIY and tools	180	2	2	11.50	0.83	3	3	2.91	0.24
40	Food and beverages	240	2	2	4.11	0.62	4	4	3.55	1.59
41	Mixed retail distr.	260	2	2	6.02	1.07	4	4	5.69	2.52
42	Flat-pack furniture	110	3	3	0.25	0.13	8	8	0.37	0.13
43	Large appliances	65	2	2	0.22	0.05	6	6	0.06	0.04
44	Office supplies	150	1	1	6.20	0.39	4	3	2.34	0.25
45	Music and hobby	90	1	1	2.12	0.09	4	3	0.58	0.15
46	Pet shop	140	2	2	3.09	1.05	7	6	1.19	0.92
47	Consumer electronics	200	2	2	3.93	0.44	4	4	2.77	0.37
48	Mixed e-commerce	280	1	1	42.04	2.08	2	2	11.85	1.95
49	Mixed retail distr.	300	2	2	11.13	1.58	4	4	11.00	4.44
50	Warehouse returns	1000	5	5	171.16	34.49	11	11	152.28	90.55

5.2 Objective value analysis

The first result is a negative one, and it is unambiguous. On the Ducato medio, the two variants return *exactly the same number of vehicles on all 50 instances*: 70 vehicles in total, with no instance on which one is better than the other. On the Torino piccolo the two variants differ on 9 instances out of 50, and on every one of them it is the FIRST-FIT variant that uses fewer vehicles: 138 vehicles in total against 147, that is $\Delta_{obj} = +6.5\%$ for the BEST-FIT variant. The exhaustive evaluation of all the admissible positions, in other words, never produced a better loading plan than stopping at the first one, and in nine cases produced a worse one.

The nine instances are `Dataset5`, `7`, `14`, `24`, `30`, `31`, `44`, `45` and `46`, and in each of them the BEST-FIT variant opens exactly one vehicle more; in the most pronounced case, `Dataset7`, it uses three vehicles against two.

This is not a defect of the implementation but a known behaviour of greedy heuristics, and it deserves an explanation. The merit function of Section 4.6 is myopic: it selects the position that wastes the least space *for the item being placed*, without any knowledge of the items that will come next. Choosing the tightest niche for the current item consumes precisely the small gaps that a later, smaller

item could have used, and it leaves the large open regions untouched; the FIRST-FIT variant, which places the item at the first admissible extreme point – and, since the list is sorted by non-decreasing z, y and x, the lowest and closest to the origin among them – tends instead to keep the load compact and to consume the free space in a more regular order. The local optimum of each single placement is not the global optimum of the loading plan, and when the vehicle is small enough for the difference to matter, the more careful rule turns out to be the worse one.

5.3 Computational time analysis

The second result is the sharpest of the whole work. The BEST-FIT variant is slower than the FIRST-FIT one on every single run, and by a wide margin: the median Δ_{time} is +883% on the Ducato medio and +448% on the Torino piccolo, with extremes ranging from +5% to +2 921%. Over the whole campaign the BEST-FIT variant spent 523 s on the Ducato medio against 74 s of the FIRST-FIT one, a factor of 7.1, and 355 s against 148 s on the Torino piccolo, a factor of 2.4.

The reason is structural. For each item and each open vehicle, the BEST-FIT variant enumerates every pair of an admissible rotation and an extreme point, and for each pair that survives the four feasibility tests it computes a merit value; the FIRST-FIT variant returns as soon as one such pair is found. The cost of a single check therefore grows with the number of extreme points, which grows with the number of items already loaded into that vehicle, and each candidate position costs a scan of those items to verify overlap and support. The result is a cost that grows more than linearly in the number of items that end up in the same vehicle.

Figure 5.1 plots the running time against the number of items on a logarithmic scale. Both variants grow steeply with the size of the instance, and the gap between them is roughly constant in relative terms over the whole range, which is what a constant multiplicative overhead looks like on a logarithmic axis. The two limit instances make the absolute magnitude concrete: on `Dataset50`, with 1 000 items, the BEST-FIT variant needs 171 s on the Ducato medio against the 34 s of the FIRST-FIT one, and 152 s against 91 s on the Torino piccolo.

One detail of Figure 5.1 is worth pausing on, because it is counter-intuitive: on the Ducato medio the BEST-FIT variant is *slower* than on the Torino piccolo, even though the Ducato medio opens far fewer vehicles. The reason is that the cost is driven by the number of items per vehicle, not by the number of vehicles. The Ducato medio swallows the whole consignment in one or two vehicles, so every new item has to be tested against a list of extreme points and of already loaded items that keeps growing; the Torino piccolo distributes the same items over several smaller vehicles, each of which stays comparatively small. The FIRST-FIT variant behaves in the opposite way, and is in fact slower on the Torino piccolo (148 s

against 74 s), because there it has to interrogate several open vehicles before finding a position, and more of those interrogations end in failure.

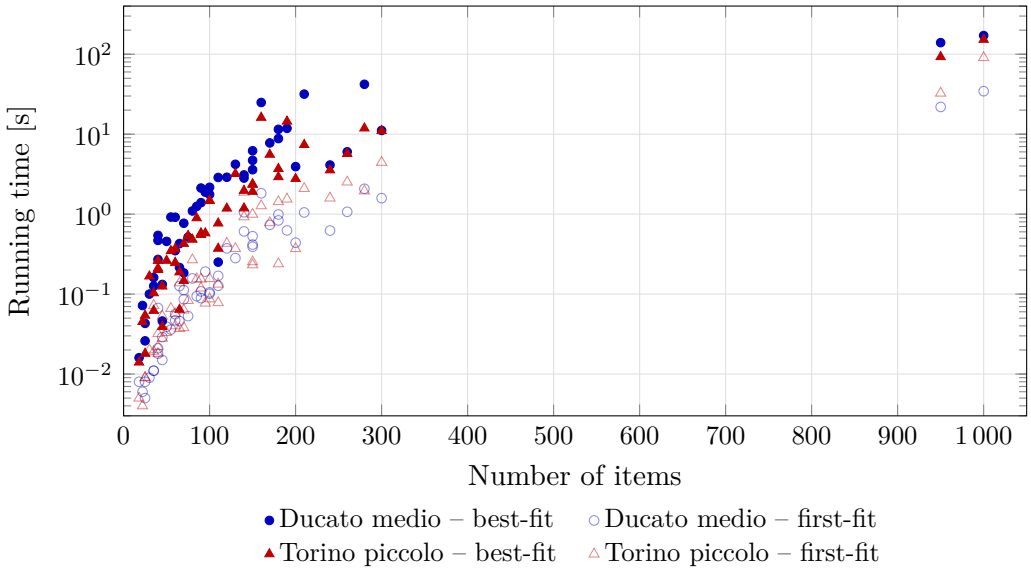


Figure 5.1: Running time against the number of items, on a logarithmic scale. Blue marks the Ducato medio and red the Torino piccolo; the solid marks are the BEST-FIT runs and the faded hollow ones the FIRST-FIT. The two BEST-FIT series lie roughly one order of magnitude above the corresponding FIRST-FIT ones over the whole range.

5.4 The effect of the vehicle size

The two vehicles were introduced to answer one question: does the gap between the two heuristics widen or narrow when the vehicle gets smaller? The answer, summarised in Table 5.2, is that the two gaps move in opposite directions.

The gap in *computing time* narrows markedly with the smaller vehicle: the median overhead of the BEST-FIT variant falls from +883% to +448%, and the ratio of the total times from 7.1 to 2.4. This follows from the mechanism described in the previous section: the smaller vehicle reduces the number of items per vehicle, which is what makes the exhaustive evaluation expensive, and at the same time increases the work of the FIRST-FIT variant, which has to try several vehicles per item. Both effects push the two variants closer together.

The gap in *solution quality*, on the contrary, only appears with the smaller vehicle. On the Ducato medio the two variants are indistinguishable, on the Torino piccolo the BEST-FIT variant is worse on nearly one instance out of five. The reason

Table 5.2: Aggregate results by vehicle over the 50 instances. LB is the sum of the continuous lower bounds of Section 5.5.

	Ducato medio	Torino piccolo
Loading volume [m^3]	11.26	2.75
Payload [kg]	1 200	660
Vehicles used, FIRST-FIT	70	138
Vehicles used, BEST-FIT	70	147
Sum of the lower bounds LB	68	110
Instances solved with one vehicle (FIRST-FIT)	37	15
Instances where BEST-FIT is worse	0	9
Total time, FIRST-FIT [s]	73.9	148.5
Total time, BEST-FIT [s]	523.4	355.0
Median Δ_{time}	+883%	+448%
Mean volume utilisation, FIRST-FIT	23.2%	47.4%

is visible in the last line of Table 5.2: on the Ducato medio the mean volume utilisation is 23%, so the loading area is never really full and the placement rule has no occasion to make a difference, whereas on the Torino piccolo it is 47%, with peaks above 70% on the two limit instances, and there the choice of the position starts to matter.

There is a further reason why the Ducato medio makes the geometry irrelevant, and it is worth stating because it was not designed on purpose. On that vehicle the binding capacity is almost never the volume but the payload: the lower bound computed on the weight exceeds the one computed on the volume on 49 instances out of 50. A vehicle of 11.26 m^3 that may carry only 1 200 kg fills up in weight long before it fills up in space, so the number of vehicles is essentially decided by the total weight of the consignment and no placement rule can improve on it; Figure 5.3 shows what such a vehicle looks like once loaded. On the Torino piccolo, whose volume was reduced four times but whose payload was reduced less than twice, the volume becomes the binding capacity on 42 instances out of 50, and the problem becomes a genuinely geometric one.

This low utilisation is not an artefact of the instances. The ratio between the payload of a vehicle and its loading volume defines the density above which the vehicle fills up in weight before it fills up in space: for the Ducato medio that threshold is 107 kg m^{-3}, while the goods of these instances average 216 kg m^{-3}, twice as much. Parcel carriers assume a comparable figure when they charge by volumetric weight, treating one cubic metre as equivalent to roughly 200 kg, which is a way of saying that ordinary goods are denser than the vans that carry them.

A van of 3.5 tonnes is built around a mass limit, and once its own tare is deducted little more than a tonne is left for eleven cubic metres; loads that exhaust the payload with the vehicle half empty are the normal condition of such a vehicle, not an exception. Practitioners distinguish the two regimes as *weighing out* and *cubing out*. At $216\,\mathrm{kg\,m^{-3}}$ the Ducato medio could not exceed 49% of volume even with a perfect packing, and the 23% actually observed is what remains once the space lost between heterogeneous boxes is taken into account. The regime does reverse when the goods are light enough: `Dataset30`, at $50\,\mathrm{kg\,m^{-3}}$, is limited by volume on both vehicles.

The practical implication is therefore double, and it is more useful than a simple ranking of the two rules. The exhaustive rule is not worth its cost when the vehicle is large relative to the consignment, because it changes nothing and costs seven times more; and it is not worth its cost when the vehicle is small either, because there it is both slower and worse. What changes with the size of the vehicle is not *whether* the BEST-FIT variant is preferable, but how expensive the mistake of choosing it is.

5.5 Distance from the lower bounds

To judge the two variants in absolute terms, and not only against each other, a lower bound on the number of vehicles is needed. Three continuous bounds are immediate: the total volume of the items divided by the loading volume, the total weight divided by the payload and the total value divided by the maximum insured value, each rounded up. Their maximum is a valid lower bound, because each of the three constraints must hold on every vehicle. The bound ignores the geometry entirely – it assumes the goods can be poured into the vehicle like a liquid – so the distance between it and a solution measures both the inefficiency of the heuristic and the unavoidable loss due to the shape of the items.

On the Ducato medio the sum of the bounds is 68 vehicles against the 70 actually used: both variants reach the bound on 48 instances out of 50, and the mean gap is 1.0%. This confirms the reading of the previous section: on that vehicle the problem is essentially not geometric, and a constructive heuristic solves it to the bound almost everywhere.

On the Torino piccolo the sum of the bounds is 110 vehicles, against the 138 of the FIRST-FIT variant and the 147 of the BEST-FIT one. The FIRST-FIT variant reaches the bound on 30 instances out of 50, with a mean gap of 24.8% against the 36.9% of the BEST-FIT variant. Part of this gap is real inefficiency and part of it is the bound being optimistic: a volume utilisation of 100% is not attainable with heterogeneous rigid boxes, and the instances where the gap is largest are those where a single vehicle would suffice by volume but the items do not interlock –

`Dataset11`, for instance, whose flat-pack furniture consists of long thin panels that fill 27% of the volume of one vehicle and still require two.

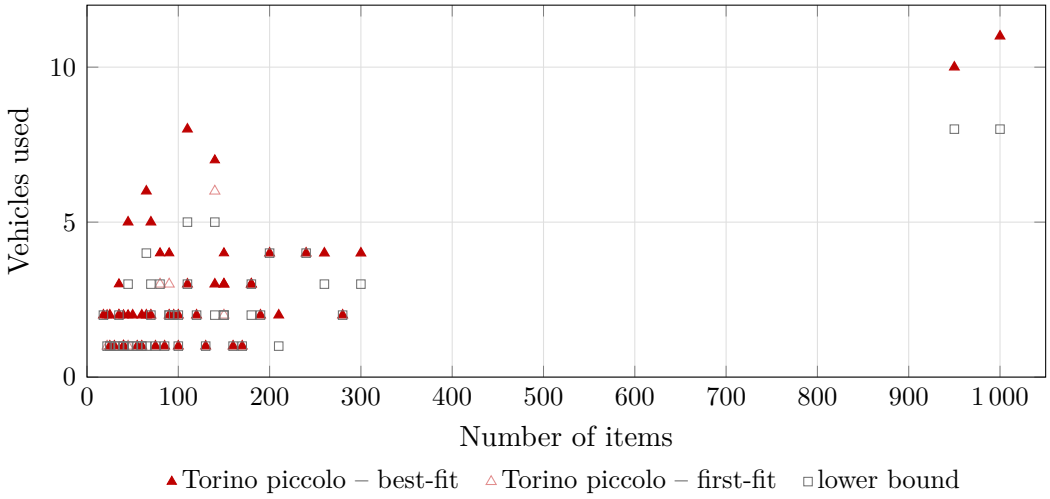


Figure 5.2: Vehicles used on the Torino piccolo against the number of items, compared with the continuous lower bound. Colours follow Figure 5.1. The three series coincide on most instances; where they separate, the BEST-FIT variant is always the highest.

Figure 5.2 shows the three series on the Torino piccolo. They coincide on most instances, which is the reason why the median gap is zero although the mean is not: the discrepancies are concentrated on a minority of instances and are of one vehicle each, but on instances whose bound is one or two vehicles a single extra vehicle is a large relative error.

5.6 Sensitivity to item ordering

The sorting by non-increasing base area is a heuristic choice like any other, and the randomization mechanism of Section 4.8 measures how much the result depends on it. Corrupting the order with probability p means that, on average, a fraction p of the items is drawn at random instead of being taken in the prescribed order.

On the Ducato medio the result is almost completely insensitive: only 2 instances out of 50 change their objective value at any level and for either variant, and of these one gets worse and one gets better. This is not a sign of robustness of the sorting rule but, again, a consequence of the Ducato medio being bound by weight: when the number of vehicles is decided by the total weight, the order in which items are loaded is irrelevant.

On the Torino piccolo 17 instances out of 50 react to the randomization: 15 of

them get worse at some level and 2 get better. Averaged over the 50 instances, the FIRST-FIT variant loses 4.8% at $p = 10\%$, 3.1% at $p = 20\%$ and 4.4% at $p = 30\%$, and the BEST-FIT variant 9.7%, 5.7% and 6.9% respectively. Two observations follow. The first is that the effect is *not monotone*: more randomization does not mean a systematically worse result, and the worst average is obtained at the lowest level of corruption. The second is that some instances actually improve, by up to 20%, when the order is perturbed. Both point at the same fact: the decreasing order is a good rule on average but not an optimal one, and on any particular instance there exist orders that beat it. Metaheuristics exploit exactly this property when they search over the sequence of items [13], which suggests that the most promising extension of this work is not a better placement rule but a search over the ordering.

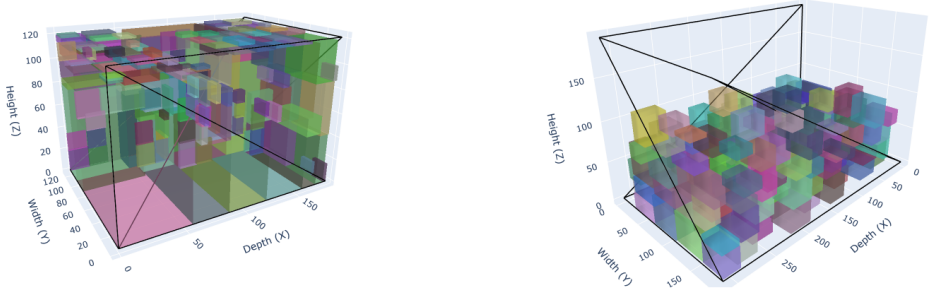
The randomization also hurts the BEST-FIT variant about twice as much as the FIRST-FIT one. A rule that optimises each single placement depends more on the items arriving in a favourable order, because it commits the best niches to whichever item happens to come first.

5.7 Packing visualization

The renderings of Section 4.9 make the numerical results legible at a glance. Figure 5.3 shows the same consignment, that of `Dataset50`, loaded by the same heuristic onto the two vehicles.

The Torino piccolo is packed to the roof, with 94% of its volume used and barely half of its payload: there the free space is reduced to the interstices between boxes, and every placement decision counts. The Ducato medio, on the right, carries a single low layer that covers the whole floor and stops at about a third of the height. That vehicle is full in the only sense that matters to it: its payload is exhausted to the last kilogram, while only 32% of its volume is occupied, and no placement rule could fill it further because any additional item would exceed the weight limit. This is why the two heuristics are indistinguishable on the larger vehicle and diverge on the smaller one: on the Ducato medio the problem is not a geometric one at all.

The renderings also show what the merit function does. Where the BEST-FIT variant opens an additional vehicle, its loads are not visibly worse than those of the FIRST-FIT variant, and are in fact locally tighter, with the items wedged into the gaps between previously placed boxes. What is missing is a regular front of free space for the items that come next, so the last ones find no room and a new vehicle has to be opened. It is the clearest illustration of the difference between a locally tight packing and a globally efficient one.



(a) Torino piccolo, vehicle 0: volume 94%, payload 51%.

(b) Ducato medio, vehicle 2: volume 32%, payload 100%.

Figure 5.3: The same consignment, that of `Dataset50`, loaded by the FIRST-FIT variant onto the two vehicles: on the left the first of the eleven Torino piccolo required, on the right the third of the five Ducato medio. The Torino piccolo is filled to the roof using half of its payload; the Ducato medio stops with two thirds of its loading area empty because it has run out of payload.

5.8 Summary of the results

The campaign gives a consistent answer across all its measurements. The FIRST-FIT variant is never worse than the BEST-FIT one in objective value: it ties on all 50 instances with the Ducato medio and wins on 9 out of 50 with the Torino piccolo. It is always faster, by a factor between 2.4 and 7.1 in aggregate and by up to thirty times on a single instance. It is more robust to a perturbation of the item ordering, and closer to the lower bound wherever the bound is attainable.

The result is stronger than the one this work set out to test. The initial question was whether the more accurate rule justified its computational cost; the answer is that on these instances it has no benefit to justify. The value of the extreme point rule, which both variants share, is confirmed, since both reach the lower bound on most instances. What is not repaid is the additional effort of ranking the candidate positions by residual space.

5.9 Managerial implications

Three practical indications follow. The first concerns the choice of the vehicle. On these consignments the Ducato medio is limited by its payload and not by its volume, so more than three quarters of its loading space travel empty on every journey; the Torino piccolo uses about twice as many journeys but fills them to 47% on average. Neither figure is satisfactory, and both point at a fleet whose capacity

is unbalanced with respect to the goods it carries: since the binding constraint of the larger vehicle is the weight and not the space, a van with the same loading volume as the Ducato medio but roughly twice its payload would cut the number of journeys without needing a single extra cubic metre.

The second concerns the choice of the algorithm. A planner who has to produce a loading plan while a customer waits cannot spend three minutes on it, and the experiments show that this is exactly what the exhaustive rule costs on a thousand-parcel consignment, with no compensation in the quality of the plan. The simpler rule produces the same plan, or a better one, in seconds.

The third concerns the constraints. The gravity strength constraint is what turns a geometrically valid plan into an executable one, and it is the test that rejects most candidate positions on stacked items. Treating it as a property of the goods rather than of the vehicle lets the same fleet carry fragile and robust goods without applying the strictest rule to everything.

5.10 Limitations and future work

The limits of this study should be stated plainly. No improvement phase follows the construction: once an item is placed it is never moved, so these are constructive solutions and nothing more. No comparison with an exact solver or with the standard benchmarks was carried out, so the distance from the optimum is known only through the continuous lower bounds, which are optimistic by construction. The fleet is homogeneous within each run, so mixing vehicles of different sizes on the same consignment, which Section 5.4 suggests could be advantageous, is not explored. Some constraints of a real loading operation are also missing, such as the distribution of the weight over the axles and the sequence in which the goods must be unloaded at successive stops.

The most significant omission is the *load bearing* constraint, that is the maximum weight each item can carry on top of itself without being crushed. The gravity strength constraint of Section 3.3 controls how much of an item's base is supported, but says nothing about how much load that item can bear in turn: nothing prevents a heavy appliance from being stacked on a box of glassware that is fully supported underneath. Adding it would require a per-item bearing capacity alongside g_i, in the form modelled by Junqueira et al. [18], and it is the first extension this work would need to be usable on real consignments.

The most promising directions follow from the results themselves. The sensitivity analysis showed that perturbing the order sometimes improves the solution, which suggests searching over the ordering rather than refining the placement rule; a randomized multi-start, which is inexpensive given how fast the FIRST-FIT variant is, would already exploit this. On the implementation side, the cost of the feasibility

check grows with the number of items in a vehicle because both the extreme points and the placed items are scanned linearly; indexing them spatially would reduce the running time of both variants and would make the exhaustive rule affordable enough to be tested on much larger instances. Finally, benchmarking against a public instance generator such as the one proposed by Osaba et al. [19] would make the results comparable with those of other authors.

Appendix A

Scripts

This appendix collects the parts of the source code that the previous chapters refer to: the generation of the instances, the loading of the input, the two sibling feasibility checks whose comparison the whole of Chapter 5 rests on, the routines that maintain the extreme points and their residual space, the main packing loop and the script that runs the experimental campaign. Auxiliary code – the data classes, the visualization module and the solution checker – is not reproduced here, since the text describes what it does and none of it takes part in the measurements.

The complete source code, together with the 50 instances and the result files discussed in Chapter 5, is available at:

`github.com/RiccardoQuariglio/RQ--Tesi-Triennale-Ricerca-Operativa-`

The listings below are extracted directly from those files, and the line numbers shown in the margin are those of the original files, so that any fragment can be located in the repository.

A.1 Instance generation

The product categories

Table A.1 reports the 19 product categories from which the items of every instance are drawn, as described in Section 4.1. Each row fixes, for one class of goods, the range of each attribute; every item of that category has its attributes drawn uniformly inside those ranges. Units are those of Section 3.1.

Table A.1: The 19 product categories used to generate the items. Rotations follow the numbering of Section 3.2; *mixed* denotes a category whose items are partly upright-only and partly freely orientable.

Category	w	d	h	ω	ν	g	R
`electronics_small`	7–20	6–16	3–9	0.15–0.9	29–1300	55–85	mixed
`electronics_large`	35–130	10–22	25–85	2–22	180–1800	75–95	01
`small_appliances`	22–55	20–45	25–40	1.5–13	25–320	55–85	01
`large_appliances`	50–65	55–70	60–90	18–75	150–650	45–70	01
`books_media`	11–26	13–20	1.5–6	0.15–1.2	7–45	10–30	012345
`toys_games`	18–50	15–40	7–20	0.3–2.5	12–160	20–45	012345
`clothing_footwear`	20–34	13–28	5–14	0.25–1.3	15–160	10–25	012345
`kitchenware_glass`	28–48	28–48	18–32	2.5–9	35–180	80–96	01
`beverages_liquids`	18–42	18–32	22–34	4–11	8–110	55–80	01
`groceries_canned`	28–45	20–32	18–28	6–12.5	9–30	15–35	012345
`furniture_flatpack`	80–160	40–70	3–8	9–32	35–260	20–40	01
`sports_fitness_light`	15–65	12–20	10–20	0.4–3	12–70	20–40	012345
`sports_fitness_heavy`	18–42	18–24	18–24	6–24	25–110	15–30	012345
`bicycles`	110–135	18–24	68–85	11–19	220–1100	65–88	0145
`tools_hardware`	20–45	10–30	8–26	1.2–13	18–220	20–45	012345
`personal_care`	10–22	8–18	6–14	0.15–0.9	18–140	65–90	01
`musical_instruments`	95–118	14–40	35–42	3–9.5	140–650	80–95	01
`pet_supplies`	35–90	25–60	12–65	3–18	20–130	20–40	012345
`office_supplies`	30–70	25–45	12–62	2.5–19	15–260	25–55	mixed

The generation script

The instances are produced by a single script with a fixed seed. The first listing shows the head of the table of categories as it appears in the code: each entry fixes the ranges of Table A.1 together with the rule that decides the admissible rotations. Five of the nineteen categories are reproduced; the others follow the same structure.

```
CATEGORIES = {
    "electronics_small": dict(
        label="Elettronica di consumo (smartphone, cuffie, action cam, tablet)",
        dims=[(7, 20), (6, 16), (3, 9)],
        weight=(0.15, 0.9), value=(29, 1300),
        gravity=(55, 85), rot_mode="mixed_free_upright", rot_upright_share=0.4,
    ),
    "electronics_large": dict(
        label="Elettronica grande (TV, monitor, laptop box)",
        dims=[(35, 130), (10, 22), (25, 85)],
```

```
        weight=(2.0, 22.0), value=(180, 1800),
        gravity=(75, 95), rot_mode="upright",
    ),
    "small_appliances": dict(
        label="Piccoli elettrodomestici (frullatori, macchine caffe', microonde, tostapane)",
        dims=[(22, 55), (20, 45), (25, 40)],
        weight=(1.5, 13.0), value=(25, 320),
        gravity=(55, 85), rot_mode="upright",
    ),
    "large_appliances": dict(
        label="Grandi elettrodomestici (frigo mini-bar, lavatrice, lavastoviglie)",
        dims=[(50, 65), (55, 70), (60, 90)],
        weight=(18.0, 75.0), value=(150, 650),
        gravity=(45, 70), rot_mode="upright",
    ),
    "books_media": dict(
        label="Libri ed editoria (tascabili, copertine rigide, cofanetti)",
        dims=[(11, 26), (13, 20), (1.5, 6)],
        weight=(0.15, 1.2), value=(7, 45),
        gravity=(10, 30), rot_mode="free",
    ),
```

Listing A.1: Product categories: ranges of dimensions, weight, value, gravity strength and rotation rule.

The second listing shows how a single item is drawn. Every attribute is sampled uniformly inside the range of its category, and the admissible rotations are assigned according to the mode of the category: fixed for the upright-only and the freely orientable ones, drawn per item for the two mixed categories, and restricted to the partial subset for boxed bicycles.

```
def sample_category_item(rng, cat_key, idx, force_rotation=None):
    """Campiona un item della categoria cat_key. force_rotation puo' essere
    'upright' o 'free' per forzare la modalita' di rotazione (usato nei
    dataset dimostrativi per mostrare entrambi i casi anche su categorie a
    modalita' mista)."""
    c = CATEGORIES[cat_key]
    (wlo, whi), (dlo, dhi), (hlo, hhi) = c["dims"]
    width = round(rng.uniform(wlo, whi), 1)
    depth = round(rng.uniform(dlo, dhi), 1)
    height = round(rng.uniform(hlo, hhi), 1)

    wlo_w, whi_w = c["weight"]
    weight = rng.uniform(wlo_w, whi_w)
    weight = round(weight, 3) if weight < 2 else round(weight, 2)

    vlo, vhi = c["value"]
```

```
    t = rng.random() ** 1.6
    value = vlo + t * (vhi - vlo)
    value = round(value, 2)

    glo, ghi = c["gravity"]
    gravity = round(rng.uniform(glo, ghi), 1)

    rot_mode = c["rot_mode"]
    if force_rotation == "upright":
        rotations = PARTIAL_ROTATIONS.get(cat_key, "01")
    elif force_rotation == "free":
        rotations = "012345"
    elif rot_mode == "free":
        rotations = "012345"
    elif rot_mode == "upright":
        rotations = PARTIAL_ROTATIONS.get(cat_key, "01")
    elif rot_mode == "mixed_free_upright":
        share = c.get("rot_upright_share", 0.3)
        rotations = "01" if rng.random() < share else "012345"
    else:
        rotations = "012345"

    return dict(
        id=f"I{idx:04d}",
        width=width, depth=depth, height=height,
        weight=weight, value=value,
        allowedRotations=rotations, gravityStrength=gravity,
    )
```

Listing A.2: Sampling of a single item from its product category.

A.2 Data loading

The two input files of an instance are read into `pandas` data frames, indexed by the identifier of the item and by the name of the vehicle. The `dtype` declaration on `allowedRotations` is necessary because the column contains strings of digits, such as `"01"`, which would otherwise be read as integers and lose their leading zero.

```
import os
import pandas as pd

# Carica un dataset (items.csv e vehicles.csv dalla cartella
    datasets/{dataset_name}/)
# in due DataFrame pandas, pronti da passare al solver.
class Instance():
    def __init__(self, dataset_name):
        self.name = dataset_name
```

```
        self.df_items = pd.read_csv(
            os.path.join('.', 'datasets', self.name, 'items.csv'),
            index_col = 0,
            dtype={"allowedRotations": str}
        )
        self.df_vehicles = pd.read_csv(
            os.path.join('.', 'datasets', self.name, 'vehicles.csv'),
            index_col = 0
        )

    def __repr__(self):
        output = ''
        output += f'--- ITEMS ---\n{self.df_items}\n'
        output += f'--- VEHICLES ---\n{self.df_vehicles}\n'
        return output
```

Listing A.3: The `Instance` class: loading of `items.csv` and `vehicles.csv`.

A.3 Core packing routines

These are the two sibling functions of Section 4.5. The first enumerates every admissible pair of a rotation and an extreme point and computes the merit of each; the second returns at the first pair that passes the tests and never computes a merit. They are deliberately kept separate, so that each variant performs exactly the work its rule requires and the comparison of running times measures the rules rather than the overhead of discarded work.

```
    @staticmethod
    def bestFitIsFeasible(item_to_pack: Item, container: Container):
        #Verifica peso e valore item (basta uno dei due a sforare per essere
    infeasible)
        if item_to_pack.weight > container.remaining_weight:
            return None
        if item_to_pack.value > container.remaining_value:
            return None

        feasible_solutions = []

        # Salvo lo stato corrente per ripristinarlo a fine funzione (l'item
    non è ancora piazzato)
        saved_rotation = item_to_pack.curr_rotation_number

        # Itero su tutte le rotazioni ammesse per questo item
        for rot_idx in item_to_pack.allowed_rotations:
            item_to_pack.setRotation(rot_idx)

            for ep in container.extreme_points:
```

```
130                 (ex, ey, ez) = ep
131                 (item_to_pack.x_position, item_to_pack.y_position,
    item_to_pack.z_position) = (ex, ey, ez)
132
133                 #Verifica limiti container (x->depth, y->width, z->height)
134                 if not (ex + item_to_pack.curr_depth <= container.depth and
135                         ey + item_to_pack.curr_width <= container.width and
136                         ez + item_to_pack.curr_height <= container.height):
137                     continue
138
139                 #Verifica sovrapposizioni con altri oggetti nel container
140                 overlapping = False
141                 for box_placed in container.items_placed:
142                     if item_to_pack.boxesOverlap(box_placed):
143                         overlapping = True
144                         break
145                 if overlapping:
146                     continue
147
148                 #Verifica Gravity Strength
149                 if ez == 0:
150                     pass  # tocca il pavimento, sempre supportato
151                 else:
152                     current_support_area = 0.0
153                     min_support_area = (item_to_pack.curr_width *
    item_to_pack.curr_depth *
154                                         (item_to_pack.gravity_strength /
    100.0))
155                     for box_placed in container.items_placed:
156                         current_support_area +=
    item_to_pack.getSupportArea(box_placed)
157                     if current_support_area + 1e-9 < min_support_area:
158                         continue
159
160                 feasible_solution = SingleSolutionFeasible(
161                     container,
162                     item_to_pack,
163                     item_width=item_to_pack.curr_width,
164                     item_depth=item_to_pack.curr_depth,
165                     item_height=item_to_pack.curr_height,
166                     ep=ep,
167                     rotation=item_to_pack.curr_rotation_number,
168                     rs=container.ep_residual_space[ep])
169                 feasible_solution.computeMerit()
170                 feasible_solutions.append(feasible_solution)
171
172         # Ripristino lo stato dell'item (non è ancora piazzato qui)
173         item_to_pack.setRotation(saved_rotation)
```

```
174         (item_to_pack.x_position, item_to_pack.y_position,
    item_to_pack.z_position) = (None, None, None)
175 
176         if not feasible_solutions:
177             return None
178         return feasible_solutions
```

Listing A.4: `bestFitIsFeasible`: exhaustive enumeration of the admissible positions, with merit.

```
180     @staticmethod
181     def firstFitIsFeasible(item_to_pack: Item, container: Container):
182         # Variante "first fit" di bestFitIsFeasible: stessa iterazione
    rotazioni/EP, ma si
183         # ferma (return) alla PRIMA soluzione fattibile trovata, e non chiama
    mai
184         # computeMerit() perché il first-fit non lo usa - calcolarlo per ogni
    combinazione
185         # sarebbe lavoro sprecato e falserebbe il confronto dei tempi di
    esecuzione con il
186         # best-fit. bestFitIsFeasible non viene toccata (resta quella usata
    dal best-fit)
187         # per non rischiare di comprometterla: la duplicazione dei controlli
188         # bounds/overlap/gravity qui sotto è voluta, non un refactor condiviso.
189         if item_to_pack.weight > container.remaining_weight:
190             return None
191         if item_to_pack.value > container.remaining_value:
192             return None
193 
194         saved_rotation = item_to_pack.curr_rotation_number
195 
196         for rot_idx in item_to_pack.allowed_rotations:
197             item_to_pack.setRotation(rot_idx)
198 
199             for ep in container.extreme_points:
200                 (ex, ey, ez) = ep
201                 (item_to_pack.x_position, item_to_pack.y_position,
    item_to_pack.z_position) = (ex, ey, ez)
202 
203                 #Verifica limiti container (x->depth, y->width, z->height)
204                 if not (ex + item_to_pack.curr_depth <= container.depth and
205                         ey + item_to_pack.curr_width <= container.width and
206                         ez + item_to_pack.curr_height <= container.height):
207                     continue
208 
209                 #Verifica sovrapposizioni con altri oggetti nel container
210                 overlapping = False
211                 for box_placed in container.items_placed:
212                     if item_to_pack.boxesOverlap(box_placed):
```

```
213                         overlapping = True
214                         break
215                 if overlapping:
216                     continue
217 
218                 #Verifica Gravity Strength
219                 if ez == 0:
220                     pass  # tocca il pavimento, sempre supportato
221                 else:
222                     current_support_area = 0.0
223                     min_support_area = (item_to_pack.curr_width *
    item_to_pack.curr_depth *
224                                         (item_to_pack.gravity_strength /
    100.0))
225                     for box_placed in container.items_placed:
226                         current_support_area +=
    item_to_pack.getSupportArea(box_placed)
227                     if current_support_area + 1e-9 < min_support_area:
228                         continue
229 
230                 # Trovata la prima soluzione fattibile: niente merito, si
    ripristina lo
231                 # stato dell'item e si ritorna subito (nessun'altra
    rotazione/EP viene provata).
232                 feasible_solution = SingleSolutionFeasible(
233                     container,
234                     item_to_pack,
235                     item_width=item_to_pack.curr_width,
236                     item_depth=item_to_pack.curr_depth,
237                     item_height=item_to_pack.curr_height,
238                     ep=ep,
239                     rotation=item_to_pack.curr_rotation_number,
240                     rs=container.ep_residual_space[ep])
241 
242                 item_to_pack.setRotation(saved_rotation)
243                 (item_to_pack.x_position, item_to_pack.y_position,
    item_to_pack.z_position) = (None, None, None)
244                 return [feasible_solution]
245 
246         # Ripristino lo stato dell'item (non è ancora piazzato qui)
247         item_to_pack.setRotation(saved_rotation)
248         (item_to_pack.x_position, item_to_pack.y_position,
    item_to_pack.z_position) = (None, None, None)
249         return None
```

Listing A.5: `firstFitIsFeasible`: same iteration, stopped at the first admissible position.

A.4 Extreme point management

The following routines implement the placement rule of Section 4.3. The function `packItemIntoContainer` loads an item, consumes the extreme point it occupied and generates the new ones; `canTakeProjection` is the predicate of Algorithm 1 of the original paper, which decides whether a corner of the item can be projected onto another item in a given direction; `shrinkResidualSpace` and `updateResidualSpace` implement Algorithm 2, the maintenance of the residual space.

```
252     @staticmethod
253     def packItemIntoContainer(solution: SingleSolutionFeasible):
254         #Variabili prese dalla soluzione
255         container = solution.container
256         item = solution.item
257 
258         #Aggiornamento Item
259         (ex, ey, ez) = solution.ep
260         (item.x_position, item.y_position, item.z_position) = (ex, ey, ez)
261         item.curr_width = solution.item_width
262         item.curr_depth = solution.item_depth
263         item.curr_height = solution.item_height
264         item.curr_rotation_number = solution.item_rotation
265 
266         #Aggiornamento Container
267         container.remaining_weight -= item.weight
268         container.remaining_value -= item.value
269         container.items_placed.append(item)
270 
271 
272         # L'EP della soluzione è stato consumato dall'item appena piazzato: va
    tolto
273         # dalla lista degli EP disponibili del container (e dal relativo RS).
274         if solution.ep in container.extreme_points:
275             container.extreme_points.remove(solution.ep)
276             del container.ep_residual_space[solution.ep]
277 
278         # Algoritmo 2 (Sez. 4.3): l'item appena piazzato può ridurre il
    Residual Space
279         # degli EP superstiti, se si frappone lungo un asse tra l'EP e ciò che
    stava tracciando.
280         PackingMethods.updateResidualSpace(container, item)
281 
282         # Per ogni direzione, tra tutti gli altri item nel container, si cerca
    quello più
283         # vicino su cui l'item appena piazzato può proiettare uno spigolo
    (canTakeProjection).
```

```
284         # max_bounds tiene il valore scalare (coordinata) del candidato
    migliore trovato finora
285         # per quella direzione (-1 = nessuno ancora); new_ex_points il punto
    3D corrispondente
286         # ((None, None, None) finché non se ne trova uno valido).
287         max_bounds = {"YX": -1,
288                       "YZ": -1,
289                       "XY": -1,
290                       "XZ": -1,
291                       "ZX": -1,
292                       "ZY": -1 }
293         new_ex_points = {"YX": (None, None, None),
294                       "YZ": (None, None, None),
295                       "XY": (None, None, None),
296                       "XZ": (None, None, None),
297                       "ZX": (None, None, None),
298                       "ZY": (None, None, None) }
299 
300         for other in container.items_placed:
301             if other == item: continue
302 
303             # --- DIREZIONI CON SUPPORTO SU Y ---
304             # YX: proiettiamo lo spigolo x (xk+dk) verso Y, cerchiamo supporto
    su faccia X
305             if PackingMethods.canTakeProjection(item, other, "YX"):
306                 potential_y = other.y_position + other.curr_width
307                 if potential_y > max_bounds["YX"]:
308                     max_bounds["YX"] = potential_y
309                     new_ex_points["YX"] = (item.x_position + item.curr_depth,
    potential_y, item.z_position)
310 
311             # YZ: proiettiamo lo spigolo z (zk+hk) verso Y, cerchiamo supporto
    su faccia Z
312             if PackingMethods.canTakeProjection(item, other, "YZ"):
313                 potential_y = other.y_position + other.curr_width
314                 if potential_y > max_bounds["YZ"]:
315                     max_bounds["YZ"] = potential_y
316                     new_ex_points["YZ"] = (item.x_position, potential_y,
    item.z_position + item.curr_height)
317 
318             # --- DIREZIONI CON SUPPORTO SU X ---
319             # XY: proiettiamo lo spigolo y (yk+wk) verso X, cerchiamo supporto
    su faccia Y
320             if PackingMethods.canTakeProjection(item, other, "XY"):
321                 potential_x = other.x_position + other.curr_depth
322                 if potential_x > max_bounds["XY"]:
323                     max_bounds["XY"] = potential_x
324                     new_ex_points["XY"] = (potential_x, item.y_position +
    item.curr_width, item.z_position)
```

```
325
326             # XZ: proiettiamo lo spigolo z (zk+hk) verso X, cerchiamo supporto
    su faccia Z
327             if PackingMethods.canTakeProjection(item, other, "XZ"):
328                 potential_x = other.x_position + other.curr_depth
329                 if potential_x > max_bounds["XZ"]:
330                     max_bounds["XZ"] = potential_x
331                     new_ex_points["XZ"] = (potential_x, item.y_position,
    item.z_position + item.curr_height)
332
333             # --- DIREZIONI CON SUPPORTO SU Z ---
334             # ZX: proiettiamo lo spigolo x (xk+dk) verso Z, cerchiamo supporto
    su faccia X
335             if PackingMethods.canTakeProjection(item, other, "ZX"):
336                 potential_z = other.z_position + other.curr_height
337                 if potential_z > max_bounds["ZX"]:
338                     max_bounds["ZX"] = potential_z
339                     new_ex_points["ZX"] = (item.x_position + item.curr_depth,
    item.y_position, potential_z)
340
341             # ZY: proiettiamo lo spigolo y (yk+wk) verso Z, cerchiamo supporto
    su faccia Y
342             if PackingMethods.canTakeProjection(item, other, "ZY"):
343                 potential_z = other.z_position + other.curr_height
344                 if potential_z > max_bounds["ZY"]:
345                     max_bounds["ZY"] = potential_z
346                     new_ex_points["ZY"] = (item.x_position, item.y_position +
    item.curr_width, potential_z)
347
348         # I 3 vertici diretti dell'item sono sempre candidati EP validi,
    indipendentemente
349         # dalla presenza di altri item (coprono il caso in cui nessun "other"
    fa da supporto
350         # in una direzione, equivalente al caso "container vuoto" del paper).
351         base_pts = [
352             (item.x_position + item.curr_depth, item.y_position,
    item.z_position),
353             (item.x_position, item.y_position + item.curr_width,
    item.z_position),
354             (item.x_position, item.y_position, item.z_position +
    item.curr_height)
355         ]
356
357         # Inserimento finale nella lista degli EP del container
358         for p in (list(new_ex_points.values()) + base_pts):
359             if p not in container.extreme_points and p != (None, None, None):
360                 # Verifichiamo i limiti fisici del bin
361                 if p[0] <= container.depth and p[1] <= container.width and
    p[2] <= container.height:
```

```
362                     container.extreme_points.append(p)
363                     # RS iniziale del nuovo EP: si parte dalle pareti del
    container e si
364                     # restringe contro ogni item già piazzato (incluso quello
    appena inserito),
365                     # per non sovrastimare lo spazio libero fin dalla
    creazione dell'EP.
366                     rs = (container.depth - p[0], container.width - p[1],
    container.height - p[2])
367                     for placed_item in container.items_placed:
368                         rs = PackingMethods.shrinkResidualSpace(p, rs,
    placed_item)
369                     container.ep_residual_space[p] = rs
370 
371         # Ordinamento EP (z, y, x non decrescente)
372         container.extreme_points.sort(key=lambda p: (p[2], p[1], p[0]))
```

Listing A.6: `packItemIntoContainer`: placement of an item and generation of the new extreme points.

```
375     @staticmethod
376     def canTakeProjection(item_new, other, direction):
377         """
378         Verifica se il punto spigolo dell'item_new può essere proiettato
379         sulla faccia dell'other nella direzione specificata.
380         """
381         # Coordinate e dimensioni item_new (nuovo)
382         x_new, y_new, z_new = item_new.x_position, item_new.y_position,
    item_new.z_position
383         w_new, d_new, h_new = item_new.curr_width, item_new.curr_depth,
    item_new.curr_height
384 
385         # Coordinate e dimensioni other (già presente)
386         x_other, y_other, z_other = other.x_position, other.y_position,
    other.z_position
387         w_other, d_other, h_other = other.curr_width, other.curr_depth,
    other.curr_height
388 
389 
390         if direction == "YX":
391             # Punto (x_new+d_new, y_new, z_new) proiettato in Y, cerchiamo
    supporto su X
392             # Requisito: Il punto deve essere nel range X e Z dell'other
393             # e trovarsi 'davanti' (y maggiore) rispetto a other
394             return (x_other <= x_new + d_new < x_other + d_other) and (z_other
    <= z_new < z_other + h_other) and (y_other + w_other <= y_new)
395 
396         elif direction == "YZ":
```

```
397             # Punto (x_new, y_new, z_new+h_new) proiettato in Y, cerchiamo
    supporto su Z
398             return (x_other <= x_new < x_other + d_other) and (y_other +
    w_other <= y_new) and (z_other <= z_new + h_new < z_other + h_other)
399
400         elif direction == "XY":
401             # Punto (x_new, y_new+w_new, z_new) proiettato in X, cerchiamo
    supporto su Y
402             return (y_other <= y_new + w_new < y_other + w_other) and (z_other
    <= z_new < z_other + h_other) and (x_other + d_other <= x_new)
403
404         elif direction == "XZ":
405             # Punto (x_new, y_new, z_new+h_new) proiettato in X, cerchiamo
    supporto su Z
406             return (y_other <= y_new < y_other + w_other) and (z_other <=
    z_new + h_new < z_other + h_other) and (x_other + d_other <= x_new)
407
408         elif direction == "ZX":
409             # Punto (x_new+d_new, y_new, z_new) proiettato in Z, cerchiamo
    supporto su X
410             return (x_other <= x_new + d_new < x_other + d_other) and (y_other
    <= y_new < y_other + w_other) and (z_other + h_other <= z_new)
411
412         elif direction == "ZY":
413             # Punto (x_new, y_new+w_new, z_new) proiettato in Z, cerchiamo
    supporto su Y
414             return (y_other <= y_new + w_new < y_other + w_other) and (x_other
    <= x_new < x_other + d_other) and (z_other + h_other <= z_new)
415
416         return False
```

Listing A.7: `canTakeProjection`: the six projection directions.

```
418     @staticmethod
419     def shrinkResidualSpace(ep_pos, rs, item):
420         """
421         Restringe (se necessario) il Residual Space di un EP considerando UN
    item già
422         piazzato: se l'item si frappone lungo un asse tra l'EP e ciò che stava
    tracciando,
423         il RS su quell'asse si riduce alla distanza dalla faccia vicina
    dell'item.
424         Ricostruzione della funzione "isOnSide" dell'Algoritmo 2 del paper
    (non definita
425         esplicitamente nel testo), derivata dalla definizione di RS come
    distanza dalla
426         parete o dall'item più vicino.
427         """
428         (ex, ey, ez) = ep_pos
```

```
429         (rs_depth, rs_width, rs_height) = rs
430         (ix, iy, iz) = (item.x_position, item.y_position, item.z_position)
431         (i_depth, i_width, i_height) = (item.curr_depth, item.curr_width,
    item.curr_height)
432 
433         # Blocco lungo depth/width: l'item deve condividere la stessa fascia
    di height dell'EP
434         if iz <= ez < iz + i_height:
435             # L'item è "davanti" all'EP lungo depth e copre la sua posizione
    lungo width?
436             if ex <= ix and iy <= ey < iy + i_width:
437                 rs_depth = min(rs_depth, ix - ex)
438             # L'item è "davanti" all'EP lungo width e copre la sua posizione
    lungo depth?
439             if ey <= iy and ix <= ex < ix + i_depth:
440                 rs_width = min(rs_width, iy - ey)
441 
442         # Blocco lungo height: l'item deve coprire l'EP sull'impronta
    depth/width
443         if ez <= iz and ix <= ex < ix + i_depth and iy <= ey < iy + i_width:
444             rs_height = min(rs_height, iz - ez)
445 
446         return (rs_depth, rs_width, rs_height)
447 
448     @staticmethod
449     def updateResidualSpace(container, item):
450         """
451         Algoritmo 2 del paper: dopo aver piazzato item, aggiorna
    incrementalmente il RS
452         di tutti gli EP superstiti del container, confrontandoli con il solo
    item appena
453         piazzato (non con l'intera storia del container: per questo resta
    economico).
454         """
455         for ep_pos in container.extreme_points:
456             container.ep_residual_space[ep_pos] =
    PackingMethods.shrinkResidualSpace(
457                 ep_pos, container.ep_residual_space[ep_pos], item)
```

Listing A.8: `shrinkResidualSpace` and `updateResidualSpace`: the residual space of an extreme point.

A.5 The packing loop

This is the implementation of Algorithm 4. The strategy is passed as a named tuple carrying the two functions that distinguish the variants, so that the loop itself is

written once and is identical for both. The parameter `random_pick_prob`, unused in the deterministic runs, implements the randomized selection of Section 4.8.

```
81      def _pack(self, strategy: _FitStrategy, vehicle_type,
        random_pick_prob=0.0, rng=None):
82          """
83          Metodo costituito da più step:
84          1) Memorizzazione degli items del dataset in una lista, e caricamento
        del container
85             template per ‘vehicle_type‘ (vedi
        loadContainer/newContainerInstance)
86          2) Sorting degli items secondo uno specifico criterio
87          3) Apertura del primo container (copia del template): da qui in avanti
        c’è il vero impaccamento
88          4) Calcolo soluzioni fattibili per ogni container aperto, ciclando su
        di essi
89          5.0 - 5.1) Si sceglie la soluzione (criterio dato da ‘strategy‘) e si
        impacca
90                     finalmente l’item, andando ad aggiornare lista_items,
        container coinvolto.
91                     Se l’item non entra in nessuno dei container già aperti, se
        ne apre uno nuovo
92                     (sempre una copia dello stesso template: non c’è scelta
        automatica tra i
93                     veicoli disponibili, ‘vehicle_type‘ è deciso dal chiamante).
94          5.2) Se anche in un container appena aperto (vuoto) l’item non entra,
        viene scartato
95               (con un warning) per evitare un loop infinito.
96
97          ‘random_pick_prob‘ (test di sensibilità all’ordinamento, non usato da
        solve()): ad ogni
98          iterazione, con questa probabilità si pesca un item a caso tra quelli
        rimanenti invece
99          di prendere sempre il prossimo in ordine (‘items_list[0]‘). Con 0.0
        (default) il
100         comportamento è identico a prima, byte per byte.
101         """
102         if random_pick_prob and rng is None:
103             rng = random.Random()
104
105         # 1.0) Memorizzazione Items
106         items_list = self.packing_methods.loadItems(self.inst.df_items)
107         # 1.1) Container template per il veicolo richiesto
108         container_template =
        self.packing_methods.loadContainer(self.inst.df_vehicles, vehicle_type)
109
110         # 2) Ordina gli item per area di base e altezza decrescente
        (euristica: i più ingombranti vengono piazzati per primi)
111         items_list = self.packing_methods.sortedItemsByAHW(items_list)
```

```
112
113         sol = self._new_sol_dict()
114         idx_vehicle = 0
115         containers_utilizzati = []
116
117         def _open_new_container():
118             nonlocal idx_vehicle
119             new_container =
    self.packing_methods.newContainerInstance(container_template)
120             new_container.idx = idx_vehicle
121             idx_vehicle += 1
122             containers_utilizzati.append(new_container)
123             return new_container
124
125         # 3) Apertura del primo container
126         if items_list:
127             _open_new_container()
128
129         #4) Per ogni container, vedere se ci sono soluzioni fattibili, e
    aggiungerli alla lista totale (delle soluz fattibili)
130         while items_list:
131             if random_pick_prob and rng.random() < random_pick_prob:
132                 pick_idx = rng.randrange(len(items_list))
133             else:
134                 pick_idx = 0
135             item_to_pack = items_list[pick_idx]
136             bool_placed = False
137             all_feasible_solutions = []
138             for container in containers_utilizzati:
139                 container_feasible_solutions =
    strategy.check_feasibility(item_to_pack, container)
140                 if container_feasible_solutions:
141                     all_feasible_solutions.extend(container_feasible_solutions)
142
143             # 5.0) Se ci sono soluzioni fattibili, sceglierne una secondo
    'strategy'
144             #      e impaccare l'item nel container
145             if all_feasible_solutions:
146                 bool_placed = True
147                 chosen_solution =
    strategy.select_solution(all_feasible_solutions)
148                 self.packing_methods.packItemIntoContainer(chosen_solution)
149                 items_list.pop(pick_idx)
150                 self._append_placement(sol, chosen_solution)
151
152             # 5.1) Se non c'è nessuna soluzione fattibile nei container già
    aperti, se ne apre uno
153             #      nuovo (copia vuota dello stesso template) e ci si prova.
154             if not bool_placed:
```

```
155                 new_container = _open_new_container()
156                 feasible_solutions = strategy.check_feasibility(item_to_pack,
    new_container)
157                 if feasible_solutions is not None:
158                     chosen_solution =
    strategy.select_solution(feasible_solutions)
159                     self.packing_methods.packItemIntoContainer(chosen_solution)
160                     bool_placed = True
161                     items_list.pop(pick_idx)
162                     self._append_placement(sol, chosen_solution)
163                 else:
164                     containers_utilizzati.pop()
165                     idx_vehicle -= 1
166 
167             if not bool_placed:
168                 # 5.2) Nemmeno un container vuoto riesce a contenere l'item:
    lo scartiamo con un
169                 # warning. Per evitare loop infinito, rimuoviamo comunque
    l'item dalla lista,
170                 # registrando che non è stato possibile piazzarlo (la
    soluzione sarà infeasible per items mancanti,
171                 # ma il solver termina invece di bloccarsi).
172                 print(f"WARNING [{strategy.name}]: l'item {item_to_pack.id}
    non è stato posizionato "
173                       f"neanche in un container vuoto.")
174                 items_list.pop(pick_idx)
175 
176         return _PackingResult(sol=sol,
    containers_utilizzati=containers_utilizzati)
```

Listing A.9: `_pack`: the constructive loop, parameterised by the strategy.

A.6 Experiment automation

The last listing is the script that produced the results of Chapter 5: for each of the 50 instances it runs the two variants on the two vehicles, measures their running times, collects the objective values at the four levels of randomization and writes the two result files per instance.

```
1 # Confronto best-fit/first-fit sui DUE veicoli fissi (furgone_ducato_medio,
2 # furgone_torino_piccolo) presenti in ogni dataset. Per ciascun dataset
    numerato
3 # (Dataset1...Dataset50) esegue 16 simulazioni (2 strategie x 4 livelli di
4 # randomicità x 2 veicoli) e scrive, nella cartella
5 # thesis_results_only_csv/Solution {dataset}/, due file:
6 #   1) {dataset}_comparison_4_simulazioni.csv   -> Δ% tempo e Δ% costo (n.
    container)
```

```
#       tra best-fit e first-fit, per ciascun veicolo, al livello 0% (non randomico).
#       4 righe: first-fit/best-fit x furgone/camion.
#   2) {dataset}_comparison_16_simulazioni.md   -> solo Δ% costo (n. container) per
#       tutte le 16 simulazioni (2 strategie x 4 livelli x 2 veicoli), rispetto al
#       first-fit non randomico dello stesso veicolo.
# solve() viene comunque chiamata (write_html=False) per ciascun veicolo: scrive
# anche i soliti CSV di dettaglio con suffisso _{vehicle_type} (comportamento
# normale, non richiesto esplicitamente ma non disattivato).
import os

from instances import Instance
from s324305_3d_packing_algorithm import s324305_3d_packing_algorithm
from generate_randomization_comparison import compute_dataset_rows

OUTPUT_DIR = 'thesis_results_only_csv'
DATASETS = [f'Dataset{i}' for i in range(1, 51)]
VEHICLES = ['furgone_ducato_medio', 'furgone_torino_piccolo']


def _fmt_number(value, decimals):
    # Formato italiano: virgola come separatore decimale
    return f'{value:.{decimals}f}'.replace('.', ',')


def _fmt_signed_pct(value):
    sign = '+' if value > 1e-9 else ('-' if value < -1e-9 else '')
    return f'{sign}{_fmt_number(abs(value), 2)}%'


def _write_4sim_csv(rows, out_path):
    import pandas as pd
    df = pd.DataFrame(rows)
    df.to_csv(out_path, index=False)


def _write_16sim_markdown(rows, out_path, dataset_name):
    headers = ['Veicolo', 'Randomicità', 'Strategia', 'Container',
               'Δ container vs first-fit 0% (stesso veicolo)']
    lines = [
        f"# Sensibilità del costo (n. container) all'ordinamento e al veicolo -- {dataset_name}",
        '',
        'Ogni riga confronta il numero di container aperti a un certo livello di',
```

```
        'randomicità dell\'ordinamento, rispetto al first-fit non randomico (0%) dello',
        'STESSO veicolo (i due veicoli non sono confrontati direttamente in questa tabella,',
        'solo tra loro stessi a livelli di randomicità diversi).',
        '',
        '| ' + ' | '.join(headers) + ' |',
        '|' + '|'.join(['---'] * len(headers)) + '|',
    ]
    for row in rows:
        level_label = f"{row['level']:.0%}"
        lines.append(
            '| ' + ' | '.join([
                row['vehicle_type'],
                level_label,
                row['strategy'],
                str(row['n_containers']),
                _fmt_signed_pct(row['delta_pct']),
            ]) + ' |'
        )
    with open(out_path, 'w', encoding='utf-8') as f:
        f.write('\n'.join(lines) + '\n')


def process_dataset(dataset_name):
    print(f"=== {dataset_name} ===", flush=True)
    inst = Instance(dataset_name)
    out_dir = os.path.join(OUTPUT_DIR, f'Solution {dataset_name}')
    os.makedirs(out_dir, exist_ok=True)

    sim4_rows = []
    sim16_rows = []

    for vehicle_type in VEHICLES:
        solver = s324305_3d_packing_algorithm(inst)
        solver.solve(write_html=False, vehicle_type=vehicle_type)

        n_best = len(solver._best_fit_containers)
        n_first = len(solver._first_fit_containers)
        t_best = solver._best_fit_time
        t_first = solver._first_fit_time

        d_cont = (n_first - n_best) / n_first * 100 if n_first else 0.0
        d_time = (t_best - t_first) / t_first * 100 if t_first else 0.0

        sim4_rows.append({
            'vehicle_type': vehicle_type, 'strategy': 'first_fit',
            'n_containers': n_first, 'execution_time_seconds': t_first,
```

```
            'delta_containers_pct_vs_firstfit': 0.0,
    'delta_time_pct_vs_firstfit': 0.0,
        })
        sim4_rows.append({
            'vehicle_type': vehicle_type, 'strategy': 'best_fit',
            'n_containers': n_best, 'execution_time_seconds': t_best,
            'delta_containers_pct_vs_firstfit': d_cont,
    'delta_time_pct_vs_firstfit': d_time,
        })
        print(f"  [{vehicle_type}] best: n={n_best} t={t_best:.3f}s | "
              f"first: n={n_first} t={t_first:.3f}s |
    delta_container={d_cont:.2f}% delta_tempo={d_time:.2f}%", flush=True)

        rows16 = compute_dataset_rows(dataset_name, vehicle_type,
    solver=solver)
        sim16_rows.extend(rows16)

    _write_4sim_csv(sim4_rows, os.path.join(out_dir,
    f'{dataset_name}_comparison_4_simulazioni.csv'))
    _write_16sim_markdown(sim16_rows, os.path.join(out_dir,
    f'{dataset_name}_comparison_16_simulazioni.md'), dataset_name)


def main():
    for dataset_name in DATASETS:
        process_dataset(dataset_name)
    print("\nFatto. Output scritto in thesis_results_only_csv/Solution
    {Dataset}/ per ogni dataset numerato.")


if __name__ == '__main__':
    main()
```

Listing A.10: `generate_vehicle_comparison.py`: the experimental campaign.

List of Tables

3.1 Mathematical notation: parameters, indexes and variables 10

4.1 The two vehicles, identical in every instance. The short names in the first column are the ones used throughout this work. 18

5.1 Complete results on the 50 instances. For each vehicle: number of vehicles used (n) and running time in seconds (t) of the two variants. 31

5.2 Aggregate results by vehicle over the 50 instances. LB is the sum of the continuous lower bounds of Section 5.5. 35

A.1 The 19 product categories used to generate the items. Rotations follow the numbering of Section 3.2; *mixed* denotes a category whose items are partly upright-only and partly freely orientable. 43

List of Figures

3.1 The six orthogonal orientations of an item, numbered as in the implementation. The reference triple is (w, d, h) with $w > d > h$. . . 12

3.2 Side view of the gravity strength constraint. The support of item i is the sum of its contact areas with the items whose top face is coplanar with its base; in case (b) the placement is rejected even though the item does not overlap anything and stays inside the vehicle. 15

4.1 Generation of extreme points. In (a) the item just placed offers three new positions along its own faces, namely $P_1 = (x_k{+}\hat{d}_k,\ y_k,\ z_k)$, $P_2 = (x_k,\ y_k{+}\hat{w}_k,\ z_k)$ and $P_3 = (x_k,\ y_k,\ z_k{+}\hat{h}_k)$; in (b) one of those positions is projected along the y axis and stops on item j, the nearest obstacle, creating a point that the corner rule would not have produced. 20

4.2 Two loading plans as rendered by the visualization module. The vehicle is drawn as a wireframe and each item as a semi-transparent solid, at the coordinates and in the rotation chosen by the heuristic. 28

5.1 Running time against the number of items, on a logarithmic scale. Blue marks the Ducato medio and red the Torino piccolo; the solid marks are the BEST-FIT runs and the faded hollow ones the FIRST-FIT. The two BEST-FIT series lie roughly one order of magnitude above the corresponding FIRST-FIT ones over the whole range. . . . 34

5.2 Vehicles used on the Torino piccolo against the number of items, compared with the continuous lower bound. Colours follow Figure 5.1. The three series coincide on most instances; where they separate, the BEST-FIT variant is always the highest. 37

5.3 The same consignment, that of `Dataset50`, loaded by the FIRST-FIT variant onto the two vehicles: on the left the first of the eleven Torino piccolo required, on the right the third of the five Ducato medio. The Torino piccolo is filled to the roof using half of its payload; the Ducato medio stops with two thirds of its loading area empty because it has run out of payload. 39

Bibliography

[1] Leonid V. Kantorovich. «Mathematical Methods of Organizing and Planning Production». In: *Management Science* 6.4 (1960), pp. 366–422 (cit. on p. 2).

[2] Paul C. Gilmore and Ralph E. Gomory. «A Linear Programming Approach to the Cutting-Stock Problem». In: *Operations Research* 9.6 (1961), pp. 849–859 (cit. on p. 2).

[3] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979 (cit. on p. 2).

[4] David S. Johnson. «Approximation Algorithms for Combinatorial Problems». In: *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing (STOC '73)*. 1973, pp. 38–49 (cit. on pp. 3, 6).

[5] Edward G. Coffman Jr., Michael R. Garey, and David S. Johnson. «Approximation Algorithms for Bin Packing: A Survey». In: *Approximation Algorithms for NP-Hard Problems*. Ed. by Dorit S. Hochbaum. Boston: PWS Publishing, 1997, pp. 46–93 (cit. on pp. 3, 6).

[6] Gerhard Wäscher, Heike Haussner, and Holger Schumann. «An Improved Typology of Cutting and Packing Problems». In: *European Journal of Operational Research* 183.3 (2007), pp. 1109–1130 (cit. on pp. 3, 6).

[7] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. «Extreme Point-Based Heuristics for Three-Dimensional Bin Packing». In: *INFORMS Journal on Computing* 20.3 (2008), pp. 368–384 (cit. on pp. 4–7, 16, 21, 25).

[8] Andreas Bortfeldt and Gerhard Wäscher. «Constraints in Container Loading — A State-of-the-Art Review». In: *European Journal of Operational Research* 229.1 (2013), pp. 1–20 (cit. on pp. 4, 6, 8).

[9] Xiaozhou Zhao, Julia A. Bennell, Tolga Bektaş, and Kathryn Dowsland. «A Comparative Review of 3D Container Loading Algorithms». In: *International Transactions in Operational Research* 23.1–2 (2016), pp. 287–320 (cit. on p. 6).

[10] Silvano Martello, David Pisinger, and Daniele Vigo. «The Three-Dimensional Bin Packing Problem». In: *Operations Research* 48.2 (2000), pp. 256–267 (cit. on p. 6).

[11] Oluf Faroe, David Pisinger, and Martin Zachariasen. «Guided Local Search for the Three-Dimensional Bin-Packing Problem». In: *INFORMS Journal on Computing* 15.3 (2003), pp. 267–283 (cit. on p. 7).

[12] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. «TS^{2}PACK: A Two-Level Tabu Search for the Three-Dimensional Bin Packing Problem». In: *European Journal of Operational Research* 195.3 (2009), pp. 744–760 (cit. on p. 7).

[13] José Fernando Gonçalves and Mauricio G. C. Resende. «A Biased Random Key Genetic Algorithm for 2D and 3D Bin Packing Problems». In: *International Journal of Production Economics* 145.2 (2013), pp. 500–510 (cit. on pp. 7, 38).

[14] Francisco Parreño, Ramón Alvarez-Valdés, José M. Tamarit, and José F. Oliveira. «A Maximal-Space Algorithm for the Container Loading Problem». In: *INFORMS Journal on Computing* 20.3 (2008), pp. 412–422 (cit. on p. 7).

[15] Riccardo Zanchetta. «3D Bin Packing: A New Heuristic Approach for Real-Case Scenarios». Master's Degree in Data Science and Engineering. Master's thesis. Politecnico di Torino, 2025 (cit. on p. 7).

[16] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. *Extreme Point-Based Heuristics for Three-Dimensional Bin Packing*. Tech. rep. CIRRELT-2007-41. Montréal, Canada: CIRRELT — Interuniversity Research Centre on Enterprise Networks, Logistics and Transportation, Oct. 2007 (cit. on pp. 7, 11).

[17] Eberhard E. Bischoff, F. Janetz, and M. S. W. Ratcliff. «Loading Pallets with Non-Identical Items». In: *European Journal of Operational Research* 84.3 (1995), pp. 681–692 (cit. on p. 8).

[18] Leonardo Junqueira, Reinaldo Morabito, and Denise Sato Yamashita. «Three-Dimensional Container Loading Models with Cargo Stability and Load Bearing Constraints». In: *Computers & Operations Research* 39.1 (2012), pp. 74–85 (cit. on pp. 8, 40).

[19] Eneko Osaba, Esther Villar-Rodriguez, and Sebastián V. Romero. «Benchmark Dataset and Instance Generator for Real-World Three-Dimensional Bin Packing Problems». In: *Data in Brief* 49 (2023), p. 109309 (cit. on pp. 8, 41).

[20] C. S. Chen, S. M. Lee, and Q. S. Shen. «An Analytical Model for the Container Loading Problem». In: *European Journal of Operational Research* 80.1 (1995), pp. 68–76 (cit. on p. 12).